

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN KẾT THÚC HỌC PHẦN
CÔNG NGHỆ PHẦN MỀM
(MSHP: 220055)

TÊN ĐỀ TÀI
BEELIFE VENTURES
NỀN TẢNG THƯƠNG MẠI ĐIỆN TỬ CHO
NGÀNH NUÔI ONG THÔNG MINH

Sinh viên thực hiện:

110122050	Trần Minh Điền	DA22TTA
110122078	Nguyễn Văn Hoàng	DA22TTD
110122061	Nguyễn Lê Duy	DA22TTD

Giáo viên hướng dẫn: TS. Nguyễn Bảo Ân

Vĩnh Long, tháng 7 năm 2025

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN KẾT THÚC HỌC PHẦN
CÔNG NGHỆ PHẦN MỀM
(MSHP: 220055)

TÊN ĐỀ TÀI

BEELIFE VENTURES
NỀN TẢNG THƯƠNG MẠI ĐIỆN TỬ CHO
NGÀNH NUÔI ONG THÔNG MINH

Sinh viên thực hiện:

110122050	Trần Minh Điền	DA22TTA
110122078	Nguyễn Văn Hoàng	DA22TTD
110122061	Nguyễn Lê Duy	DA22TTD

Giáo viên hướng dẫn: TS. Nguyễn Bảo Ân

LỜI CAM ĐOAN

Chúng em xin cam đoan đây là công trình nghiên cứu và phát triển của nhóm chúng em, được thực hiện trong khuôn khổ đề án môn học Công nghệ Phần mềm dưới sự hướng dẫn của thầy **TS. Nguyễn Bảo Ân**.

Các nội dung, số liệu và kết quả được trình bày trong báo cáo là trung thực, khách quan và là sản phẩm từ quá trình làm việc của nhóm. Toàn bộ các nguồn tài liệu tham khảo và sự giúp đỡ trong quá trình thực hiện đều đã được trích dẫn và ghi nhận đầy đủ.

Công trình này chưa từng được công bố dưới bất kỳ hình thức nào trước đây.

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa, việc xây dựng các hệ thống phần mềm không chỉ đòi hỏi kiến thức về công nghệ mà còn cần một quy trình phát triển chuyên nghiệp, có hệ thống để đảm bảo sản phẩm cuối cùng đạt chất lượng cao, dễ bảo trì và có khả năng mở rộng. Môn học **Công nghệ Phần mềm** cung cấp nền tảng lý thuyết và các phương pháp luận cốt lõi để giải quyết bài toán này, từ khâu phân tích yêu cầu, thiết kế kiến trúc, triển khai, kiểm thử cho đến quản lý dự án.

Để ứng dụng các kiến thức đã học vào một bài toán thực tế, nhóm chúng tôi đã chọn thực hiện đề án **“BeeLife Ventures – Nền tảng thương mại điện tử cho ngành nuôi ong thông minh”**. Đề tài này không chỉ nhằm mục đích xây dựng một ứng dụng có giá trị thực tiễn, mà còn là cơ hội để chúng em áp dụng toàn diện quy trình phát triển phần mềm hiện đại.

Trong khuôn khổ đề án, chúng em đã thực hiện đầy đủ các giai đoạn của một vòng đời phát triển phần mềm. Bắt đầu từ việc **phân tích yêu cầu** (functional & non-functional), chúng em tiến hành **thiết kế hệ thống** với kiến trúc Client-Server, lựa chọn các **mẫu thiết kế (Design Patterns)** phù hợp như Repository, Service, DTO để đảm bảo mã nguồn có cấu trúc rõ ràng và linh hoạt. Quá trình **quản lý dự án** được thực hiện theo phương pháp Agile/Scrum, sử dụng công cụ Jira để lập kế hoạch, theo dõi tiến độ qua các Sprint và phân công nhiệm vụ cụ thể. Cuối cùng, giai đoạn **kiểm thử và triển khai tự động (CI/CD)** với GitHub Actions và Docker được chú trọng để đảm bảo chất lượng và tính ổn định của sản phẩm.

Báo cáo này sẽ trình bày chi tiết toàn bộ quá trình thực hiện dự án, không chỉ về kết quả sản phẩm mà còn tập trung vào các quyết định kỹ thuật, các thách thức gặp phải và những bài học kinh nghiệm về công nghệ phần mềm mà nhóm đã đúc kết được.

LỜI CẢM ƠN

Đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến quý thầy cô Trường Đại học Trà Vinh và đặc biệt là quý thầy cô Khoa Kỹ thuật và Công nghệ đã trang bị cho chúng em những kiến thức nền tảng vững chắc trong suốt quá trình học tập.

Chúng em xin bày tỏ lòng biết ơn sâu sắc nhất đến thầy **TS. Nguyễn Bảo Ân**, người đã tận tình hướng dẫn, định hướng và đưa ra những góp ý chuyên môn quý báu trong suốt thời gian nhóm chúng em thực hiện đồ án môn học **Công nghệ Phần mềm**. Nhờ sự chỉ bảo của thầy về các phương pháp luận, quy trình phát triển, và các kỹ thuật quản lý dự án, chúng em đã có thể áp dụng lý thuyết vào thực tiễn một cách hiệu quả và hoàn thành đề tài này.

Thông qua đồ án, chúng em không chỉ xây dựng được một sản phẩm phần mềm mà còn tích lũy được nhiều kinh nghiệm thực tế về làm việc nhóm, phân tích thiết kế hệ thống và quản lý một dự án từ đầu đến cuối. Mặc dù đã rất cố gắng, nhưng do kiến thức và thời gian có hạn, báo cáo không thể tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp của quý thầy cô để đề tài được hoàn thiện hơn.

Cuối cùng, chúng em xin cảm ơn gia đình, bạn bè đã luôn động viên, ủng hộ để chúng em có điều kiện học tập và hoàn thành tốt đồ án.

Chúng em xin chân thành cảm ơn!

Của giảng viên hướng dẫn đồ án

Giảng viên hướng dẫn
(ký và ghi rõ họ tên)

Của giảng viên phản biện đề án thứ nhất

Giảng viên phản biện
(ký và ghi rõ họ tên)

Của giảng viên phản biện đề án thứ hai

Giảng viên phản biện
(ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU	1
1.1. Tên dự án và chủ đề	1
1.2. Mục tiêu của ứng dụng	1
1.3. Lý do chọn đề tài	1
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU	3
2.1. Các chức năng chính của hệ thống	3
2.1.1. Quản lý người dùng	3
2.1.2. Quản lý sản phẩm	3
2.1.3. Giỏ hàng và Đơn hàng	3
2.1.4. Quản trị hệ thống	3
2.1.5. Tích hợp dịch vụ bên ngoài	3
2.2. Các yêu cầu phi chức năng	4
2.2.1. Hiệu năng	4
2.2.2. Bảo mật	4
2.2.3. Khả năng mở rộng	4
2.2.4. Độ tin cậy	4
2.2.5. Khả năng sử dụng	4
2.2.6. Khả năng bảo trì	4
Chương 3: THIẾT KẾ HỆ THỐNG	5
3.1. Kiến trúc tổng thể	5
3.1.1. Mô hình kiến trúc	5
3.1.2. Các thành phần chính	6
3.2. Thiết kế cơ sở dữ liệu	8
3.2.1. Mô hình Entity-Relationship Diagram (ERD)	8
3.2.2. Mô tả các entity chính	9

3.3. Thiết kế API	10
3.3.1. RESTful API Architecture	10
3.3.2. API Security	11
3.3.3. API Documentation	11
3.4. Thiết kế giao diện (UI/UX)	11
3.4.1. Design System	11
3.4.2. Responsive Design	12
3.5. Design Patterns trong Backend	13
3.5.1. Repository Pattern	13
3.5.2. Service Pattern	14
3.5.3. DTO Pattern	14
3.5.4. Dependency Injection	14
3.5.5. Singleton Pattern	14
3.5.6. Factory Pattern	15
3.5.7. Transaction Pattern	15
3.5.8. Adapter Pattern	15
3.5.9. Strategy Pattern	15
CHƯƠNG 4: TRIỂN KHAI CÔNG NGHỆ SỬ DỤNG	16
4.1. Danh sách các công nghệ đã sử dụng	16
4.1.1. Công nghệ phía giao diện người dùng	16
4.1.2. Công nghệ phía máy chủ	16
4.1.3. API và dịch vụ bên ngoài	17
4.2. Quy trình CI/CD với GitHub Actions	17
4.2.1. Workflow Structure	17
4.2.2. Chiến lược triển khai	19
4.3. Cấu hình Docker và quy trình triển khai ứng dụng	19

4.3.1. Docker Configuration	19
4.3.2. Docker Compose Setup	20
4.3.3. Production Deployment	20
4.3.4. Environment Management	21
CHƯƠNG 5: QUẢN LÝ DỰ ÁN	22
5.1. Cách sử dụng Jira để lập kế hoạch và theo dõi tiến độ	22
5.1.1. Thiết lập Project trong Jira	22
5.1.2. Kế hoạch Jira	22
5.1.3. Lập kế hoạch Sprint	25
5.1.4. Phân tích biểu đồ Burndown	26
5.2. Phân công nhiệm vụ của từng thành viên trong nhóm	26
5.2.1. Cấu trúc Team và Roles	26
5.2.2. Chi tiết phân công	27
5.2.3. Quy trình cộng tác	28
CHƯƠNG 6: KIỂM THỬ	30
6.1. Chiến lược kiểm thử và công cụ sử dụng	30
6.1.1. Testing Strategy Overview	30
6.1.2. Frontend Testing	30
6.1.3. Backend Testing	31
6.1.4. Kiểm thử API với Postman	32
6.2. Kết quả kiểm thử API	33
6.2.1. Authentication API Testing	33
6.2.2. Product API Testing	35
6.2.3. Cart API Testing	37
6.2.4. Order API Testing	38
6.2.5. Admin API Testing	39

6.2.6. Tự động hóa Kiểm thử với GitHub Actions	39
CHƯƠNG 7: ĐÁNH GIÁ VÀ KẾT LUẬN	41
7.1. Những khó khăn gặp phải trong quá trình thực hiện	41
7.1.1. Khó khăn	41
7.1.2. Khó khăn về Project Management	42
7.1.3. Khó khăn về Testing & Deployment	43
7.2. Bài học rút ra và đề xuất cải thiện trong tương lai	43
7.2.1. Bài Học Kỹ Thuật Rút Ra	43
7.2.2. Những Bài Học Quản Lý Dự Án	44
7.2.3. Đề Xuất Cải Thiện Tương Lai	45
CHƯƠNG 8: PHỤ LỤC	47
8.1. Hướng dẫn cài đặt và chạy ứng dụng	47
8.1.1. Yêu cầu hệ thống	47
8.1.2. Thiết lập phát triển cục bộ	47
8.1.3. Triển khai bằng Docker	48
8.1.4. Triển khai thực tế (Production)	48
8.2. GitHub Repository & Liên kết bản demo	50
8.2.1. Kho lưu trữ GitHub (GitHub Repository)	50
8.2.2. Liên kết demo trực tiếp (Live Demo Links)	51
8.2.3. Liên kết tài liệu (Documentation Links)	51
8.2.4. Giám sát & Phân tích (Monitoring & Analytics)	52
8.2.5. Hỗ trợ & Liên hệ (Support & Contact)	52
TÀI LIỆU THAM KHẢO	53

DANH MỤC CÁC BẢNG

Bảng 2: Kế hoạch Jira.....	22
----------------------------	----

DANH MỤC CÁC HÌNH

Hình 1: Mô hình kiến trúc hệ thống	5
Hình 2: Mô hình Entity-Relationship Diagram (ERD)	8
Hình 3: Cấu trúc thư mục RESTful API	10
Hình 4: Kết trúc component	12
Hình 5: cấu trúc thành viên nhóm dự án (Project Team)	26
Hình 6: Kết quả kiểm thử API POST /api/auth/register.....	34
Hình 7: Kết quả kiểm thử API POST /api/auth/login	35
Hình 8: Kết quả kiểm thử API GET /api/product.....	36
Hình 9: Kết quả kiểm thử API GET /api/product/{id}	37
Hình 10: Kết quả kiểm thử API GET /api/cart.....	37
Hình 11: Kết quả kiểm thử API POST /api/orders.....	38
Hình 12: Kết quả kiểm thử API GET /api/admin/dashboard	39

DANH SÁCH TỪ VIẾT TẮT

Từ Viết Tắt	Từ Đầy Đủ
AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
CI/CD	Continuous Integration / Continuous Deployment (Tích hợp liên tục / Triển khai liên tục)
CRUD	Create, Read, Update, Delete (Các thao tác cơ bản: Tạo, Đọc, Cập nhật, Xóa)
CSDL	Cơ sở dữ liệu
CSRF	Cross-Site Request Forgery (Tấn công giả mạo yêu cầu liên trang)
DTO	Data Transfer Object (Đối tượng truyền dữ liệu)
ERD	Entity-Relationship Diagram (Lược đồ quan hệ thực thể)
HTML	HyperText Markup Language (Ngôn ngữ đánh dấu siêu văn bản)
HTTP	Hypertext Transfer Protocol (Giao thức truyền tải siêu văn bản)
JPA	Jakarta Persistence API (Giao diện lập trình bền vững Jakarta)
JSON	JavaScript Object Notation (Ký pháp đối tượng JavaScript)
JWT	JSON Web Token (Mã thông báo web JSON)
ORM	Object-Relational Mapping (Ánh xạ đối tượng - quan hệ)
SQL	Structured Query Language (Ngôn ngữ truy vấn có cấu trúc)
SSR	Server-Side Rendering (Kết xuất phía máy chủ)
TMĐT	Thương mại điện tử
UI	User Interface (Giao diện người dùng)
URL	Uniform Resource Locator (Định vị tài nguyên thống nhất)
UX	User Experience (Trải nghiệm người dùng)
VPS	Virtual Private Server (Máy chủ ảo cá nhân)

AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
CI/CD	Continuous Integration / Continuous Deployment (Tích hợp liên tục / Triển khai liên tục)
CRUD	Create, Read, Update, Delete (Các thao tác cơ bản: Tạo, Đọc, Cập nhật, Xóa)
CSDL	Cơ sở dữ liệu
CSRF	Cross-Site Request Forgery (Tấn công giả mạo yêu cầu liên trang)
DTO	Data Transfer Object (Đối tượng truyền dữ liệu)
ERD	Entity-Relationship Diagram (Lược đồ quan hệ thực thể)
HTML	HyperText Markup Language (Ngôn ngữ đánh dấu siêu văn bản)
HTTP	Hypertext Transfer Protocol (Giao thức truyền tải siêu văn bản)
JPA	Jakarta Persistence API (Giao diện lập trình bền vững Jakarta)
JSON	JavaScript Object Notation (Ký pháp đối tượng JavaScript)
JWT	JSON Web Token (Mã thông báo web JSON)
ORM	Object-Relational Mapping (Ánh xạ đối tượng - quan hệ)
SQL	Structured Query Language (Ngôn ngữ truy vấn có cấu trúc)
SSR	Server-Side Rendering (Kết xuất phía máy chủ)
TMĐT	Thương mại điện tử
UI	User Interface (Giao diện người dùng)
URL	Uniform Resource Locator (Định vị tài nguyên thống nhất)
UX	User Experience (Trải nghiệm người dùng)
VPS	Virtual Private Server (Máy chủ ảo cá nhân)

CHƯƠNG 1: GIỚI THIỆU

1.1. Tên dự án và chủ đề

BeeLife Ventures là một nền tảng thương mại điện tử chuyên về các sản phẩm và giải pháp nuôi ong thông minh. Dự án được phát triển nhằm hiện đại hóa ngành nuôi ong tại Việt Nam thông qua việc ứng dụng công nghệ thông tin và phần mềm dựa trên đám mây.

1.2. Mục tiêu của ứng dụng

- **Số hóa ngành nuôi ong:** Chuyển đổi số quy trình mua bán sản phẩm nuôi ong từ truyền thống sang hiện đại
- **Kết nối cộng đồng:** Tạo cầu nối giữa người nuôi ong, nhà cung cấp thiết bị và khách hàng
- **Quản lý thông minh:** Cung cấp giải pháp quản lý đơn hàng, sản phẩm và khách hàng hiệu quả
- **Mở rộng thị trường:** Giúp các doanh nghiệp trong ngành nuôi ong tiếp cận khách hàng rộng hơn
- **Nâng cao trải nghiệm:** Cung cấp giao diện thân thiện và tính năng hiện đại cho người dùng

1.3. Lý do chọn đề tài

Việt Nam là một trong những quốc gia có truyền thống nuôi ong lâu đời với tiềm năng phát triển lớn. Tuy nhiên, ngành này vẫn chưa được ứng dụng công nghệ thông tin một cách hiệu quả. Những lý do chính để chọn đề tài này:

Nhu cầu thực tiễn:

- Thị trường nuôi ong Việt Nam đang phát triển mạnh mẽ
- Thiếu các nền tảng chuyên biệt cho ngành nuôi ong
- Nhu cầu số hóa quy trình kinh doanh trong ngành

Tính khả thi công nghệ:

- Áp dụng được các công nghệ hiện đại như microservices, cloud computing
- Phù hợp với xu hướng chuyển đổi số
- Có thể tích hợp AI/IoT cho giám sát đàn ong trong tương lai

Giá trị kinh tế và xã hội:

- Hỗ trợ nông dân và các hộ nuôi ong nâng cao thu nhập
- Góp phần bảo vệ môi trường thông qua nuôi ong bền vững
- Tạo ra chuỗi giá trị hoàn chỉnh từ sản xuất đến tiêu thụ

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU

2.1. Các chức năng chính của hệ thống

2.1.1. Quản lý người dùng

- **Đăng ký tài khoản:** Người dùng có thể tạo tài khoản mới với thông tin cá nhân
- **Đăng nhập/Đăng xuất:** Xác thực người dùng qua JWT token
- **Quản lý hồ sơ:** Cập nhật thông tin cá nhân, địa chỉ, số điện thoại
- **Phân quyền:** Hệ thống USER và ADMIN với các quyền hạn khác nhau

2.1.2. Quản lý sản phẩm

- **Hiển thị sản phẩm:** Danh sách sản phẩm với thông tin chi tiết, hình ảnh, giá cả
- **Tìm kiếm sản phẩm:** Tìm kiếm theo tên, loại sản phẩm, khoảng giá
- **Phân loại sản phẩm:** Chia sản phẩm theo danh mục (thiết bị nuôi ong, mật ong, phụ kiện)
- **Quản lý kho:** Theo dõi số lượng tồn kho, cảnh báo hết hàng

2.1.3. Giỏ hàng và Đơn hàng

- **Thêm vào giỏ hàng:** Người dùng có thể thêm sản phẩm vào giỏ hàng
- **Quản lý giỏ hàng:** Cập nhật số lượng, xóa sản phẩm, tính tổng tiền
- **Đặt hàng:** Tạo đơn hàng từ giỏ hàng với thông tin giao hàng
- **Theo dõi đơn hàng:** Xem trạng thái đơn hàng (Pending → Processing → Shipped → Delivered)

2.1.4. Quản trị hệ thống

- **Dashboard:** Hiển thị thống kê tổng quan về doanh thu, đơn hàng, người dùng
- **Quản lý sản phẩm:** CRUD operations cho sản phẩm
- **Quản lý đơn hàng:** Cập nhật trạng thái, xử lý đơn hàng
- **Quản lý người dùng:** Xem danh sách, khóa/mở khóa tài khoản
- **Báo cáo doanh thu:** Xuất báo cáo theo khoảng thời gian

2.1.5. Tích hợp dịch vụ bên ngoài

- **Google Drive API:** Upload và quản lý hình ảnh sản phẩm
- **Gemini AI Chatbot:** Hỗ trợ khách hàng 24/7

2.2. Các yêu cầu phi chức năng

2.2.1. Hiệu năng

- **Thời gian phản hồi:** < 2 giây cho các trang chính
- **Thông lượng:** Hỗ trợ đồng thời 1000+ người dùng
- **Tải trang:** Tối ưu hóa hình ảnh và code splitting

2.2.2. Bảo mật

- **Xác thực:** JWT token với thời gian hết hạn
- **Mã hóa mật khẩu:** BCrypt encryption
- **HTTPS:** Bảo mật truyền tải dữ liệu
- **CORS:** Cấu hình cross-origin resource sharing
- **Input validation:** Kiểm tra dữ liệu đầu vào

2.2.3. Khả năng mở rộng

- **Kiến trúc microservices:** Có thể mở rộng từng service độc lập
- **Database sharding:** Khả năng phân tán dữ liệu
- **Load balancing:** Cân bằng tải giữa các instance

2.2.4. Độ tin cậy

- **Uptime:** 99.9% thời gian hoạt động
- **Backup:** Sao lưu dữ liệu định kỳ
- **Health checks:** Giám sát sức khỏe hệ thống

2.2.5. Khả năng sử dụng

- **Responsive design:** Hoạt động tốt trên mọi thiết bị
- **UX/UI hiện đại:** Giao diện thân thiện, dễ sử dụng
- **Đa ngôn ngữ:** Hỗ trợ tiếng Việt và tiếng Anh
- **Accessibility:** Tuân thủ các tiêu chuẩn web accessibility

2.2.6. Khả năng bảo trì

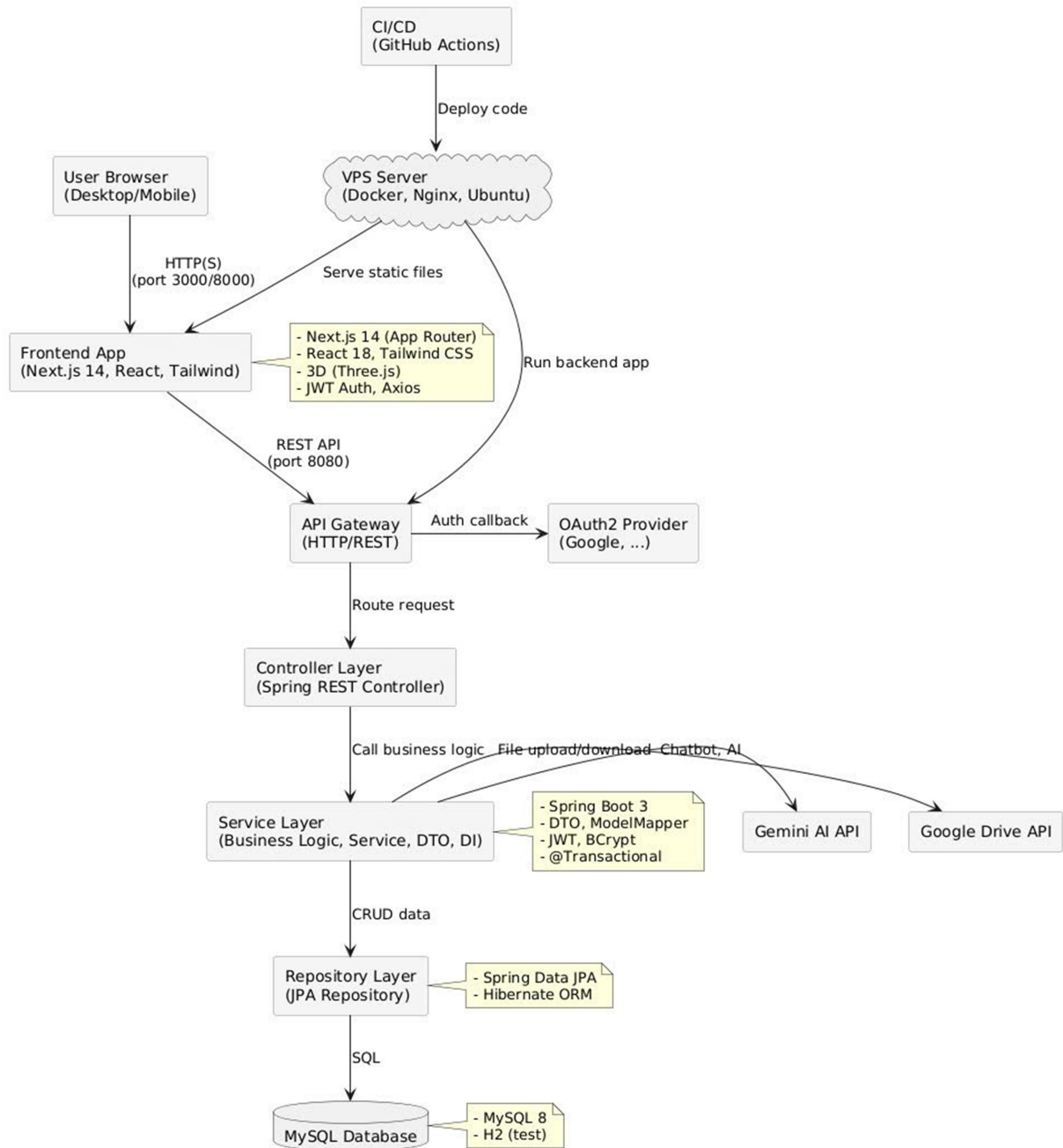
- **Code quality:** Tuân thủ coding standards
- **Documentation:** Tài liệu API đầy đủ
- **Testing:** Unit tests và integration tests
- **CI/CD:** Tự động hóa quá trình deployment

Chương 3: THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể

3.1.1. Mô hình kiến trúc

BeeLife Ventures hiện tại được thiết kế theo mô hình **Monolithic Client-Server Architecture** (đa tầng, client-server), gồm các thành phần chính:



Hình 1: Mô hình kiến trúc hệ thống

- **Frontend:** Next.js (React) phục vụ giao diện người dùng, gọi API backend qua HTTP.
- **Backend:** Spring Boot (Java) là một ứng dụng monolith, xử lý toàn bộ business logic, xác thực, quản lý dữ liệu, kết nối database và các dịch vụ ngoài.
- **Database:** MySQL lưu trữ dữ liệu tập trung.
- **External APIs:** Tích hợp Google Drive, Gemini AI, OAuth2.

Dự án định hướng có thể tách thành microservices trong tương lai (ví dụ: tách service cho user, order, product), nhưng hiện tại toàn bộ backend là một ứng dụng monolithic duy nhất để đơn giản hóa triển khai, bảo trì và phù hợp quy mô nhóm.

3.1.2. Các thành phần chính

Frontend Layer:

- **Next.js 14:** Một framework dựa trên React hỗ trợ kết xuất phía máy chủ (Server-Side Rendering).
- **TypeScript:** Ngôn ngữ lập trình bổ sung kiểm tra kiểu dữ liệu, giúp tăng độ tin cậy và trải nghiệm phát triển.
- **Tailwind CSS:** Framework CSS với phương pháp tiếp cận "tiện ích trước" (utility-first) để xây dựng giao diện nhanh chóng.
- **Three.js:** Thư viện JavaScript dùng để hiển thị mô hình sản phẩm 3D trên trình duyệt.

Backend Layer:

- **Spring Boot 3:** Ứng dụng dạng đơn khối (monolith), tổ chức theo mô hình nhiều tầng gồm: Controller - Service - Repository.
- **Spring Security:** Cung cấp chức năng xác thực và phân quyền truy cập.
- **Spring Data JPA:** Hỗ trợ ánh xạ đối tượng - quan hệ (ORM) giữa Java và cơ sở dữ liệu.
- **JWT (JSON Web Token):** Phương thức xác thực không trạng thái (stateless authentication) dùng trong API.

Database Layer:

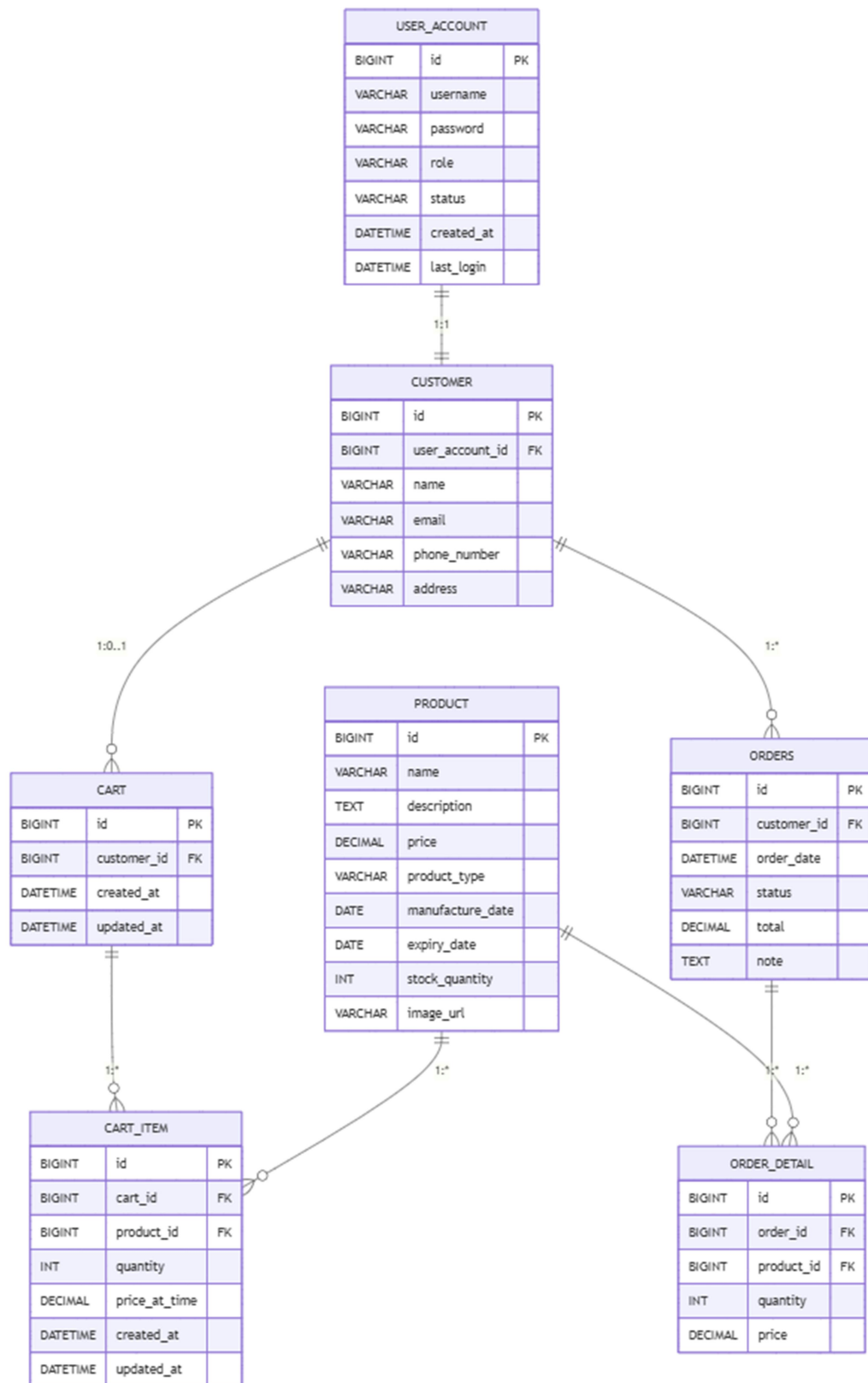
- **MySQL 8:** Cơ sở dữ liệu quan hệ chính, dùng để lưu trữ dữ liệu hệ thống.
- **H2 Database:** Cơ sở dữ liệu trong bộ nhớ, dùng cho mục đích kiểm thử (testing).

Infrastructure Layer:

- **Docker:** Công cụ đóng gói ứng dụng và các thành phần liên quan vào container.
- **GitHub Actions:** Công cụ tích hợp và triển khai liên tục (CI/CD pipeline).
- **VPS (Máy chủ riêng ảo):** Được sử dụng để triển khai ứng dụng lên môi trường đám mây.

3.2. Thiết kế cơ sở dữ liệu

3.2.1. Mô hình Entity-Relationship Diagram (ERD)



Hình 2: Mô hình Entity-Relationship Diagram (ERD)

3.2.2. Mô tả các entity chính

USER_ACCOUNT:

- Lưu trữ thông tin xác thực và đăng nhập.
- Áp dụng cơ chế phân quyền dựa trên vai trò (USER/ADMIN).
- Ghi nhận thời gian đăng nhập gần nhất để phục vụ phân tích hành vi người dùng.

CUSTOMER:

- Lưu thông tin hồ sơ cá nhân của người dùng.
- Có mối quan hệ một - một (one-to-one) với bảng USER_ACCOUNT.

PRODUCT:

- Danh mục sản phẩm đầy đủ thông tin mô tả (metadata).
- Quản lý tồn kho, bao gồm theo dõi số lượng.
- Hỗ trợ nhiều loại sản phẩm liên quan đến nuôi ong.

CART & CART_ITEM:

- Giỏ hàng được lưu trữ vĩnh viễn giữa các phiên làm việc.
- Ghi nhận giá tại thời điểm người dùng thêm sản phẩm vào giỏ (price snapshot).

ORDERS & ORDER_DETAIL:

- Quản lý đơn hàng với các trạng thái xử lý theo quy trình (workflow).
- Lưu thông tin chi tiết từng sản phẩm trong đơn, bao gồm lịch sử giá tại thời điểm đặt hàng.

3.3. Thiết kế API

3.3.1. RESTful API Architecture

API được thiết kế theo chuẩn REST với structure như sau:

```
/api/v1/
├── /auth/                                # Authentication endpoints
│   ├── POST /register                    # User registration
│   ├── POST /login                      # User login
│   ├── GET /profile                     # Get user profile
│   └── PUT /profile                     # Update profile
├── /products/                           # Product endpoints
│   ├── GET /                            # Get all products
│   └── GET /{id}                        # Get product by ID
├── /cart/                               # Shopping cart endpoints
│   ├── GET /                            # Get user cart
│   ├── POST /add                        # Add to cart
│   ├── PUT /update                      # Update cart item
│   ├── DELETE /remove/{id}             # Remove from cart
│   ├── DELETE /clear                   # Clear cart
│   ├── GET /count                      # Get cart items count
│   └── POST /checkout                  # Create order from cart
├── /orders/                             # Order management endpoints
│   ├── GET /                            # Get user orders
│   ├── GET /{id}                       # Get order by ID
│   ├── POST /                          # Create order
│   ├── PUT /                           # Update order
│   └── DELETE /{id}                    # Delete order
└── /admin/                              # Admin endpoints
    ├── GET /dashboard                  # Admin dashboard
    ├── /users/                         # User management
    ├── /products/                     # Product management
    ├── /orders/                       # Order management
    └── /revenue/report                 # Revenue reports
```

Hình 3: Cấu trúc thư mục RESTful API

3.3.2. API Security

JWT Authentication:

Authorization: Bearer <jwt_token>

Cấu hình CORS:

origins: "*"

maxAge: 3600

Xử lý dữ liệu đầu vào:

- **Jakarta Bean Validation:** Sử dụng các annotation tiêu chuẩn để kiểm tra dữ liệu đầu vào của người dùng (ví dụ: @NotNull, @Size, @Email...).
- **Custom validation annotations:** Xây dựng các annotation kiểm tra dữ liệu tùy chỉnh theo yêu cầu nghiệp vụ cụ thể.
- **Xử lý lỗi với ResponseEntity:** Trả về thông báo lỗi chi tiết cho client thông qua đối tượng ResponseEntity, đảm bảo API thân thiện và dễ hiểu.

3.3.3. API Documentation

Sử dụng **Swagger/OpenAPI 3** để tự động generate documentation:

- Endpoint: <http://localhost:8080/swagger-ui/index.html>
- Real-time testing interface
- Schema definitions và examples

3.4. Thiết kế giao diện (UI/UX)

3.4.1. Design System

Bảng màu:

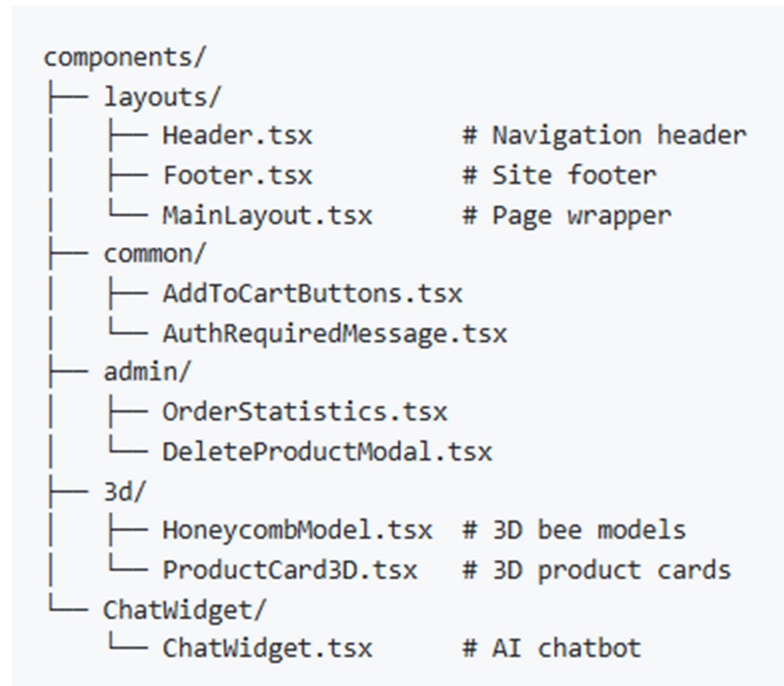
- **Màu chính** : #65BD60 – Xanh lá cây, đại diện cho thiên nhiên và hình ảnh của loài ong.
- **Màu phụ** : #4E4540 – Nâu đất, thể hiện sự mộc mạc, truyền thống.
- **Nền** : #ECF1E5 – Xanh nhạt, tạo cảm giác nhẹ nhàng và dễ chịu.
- **Văn bản** : #1a202c – Xám đậm, đảm bảo độ tương phản và dễ đọc.

Kiểu chữ :

- **Font chữ sử dụng:** Inter và các font hệ thống (system fonts).

- **Kích thước chữ phản hồi** : Được điều chỉnh linh hoạt theo thiết bị bằng Tailwind CSS.
- **Cấu trúc phân cấp rõ ràng**: Sử dụng các cấp độ tiêu đề (heading) để tạo bố cục thông tin dễ theo dõi.

Component Architecture:



Hình 4: Kết cấu component

3.4.2. Responsive Design

Breakpoints:

- Mobile: < 768px
- Tablet: 768px - 1024px
- Desktop: > 1024px

Giao diện mobile:

- **Giao diện thân thiện với cảm ứng**: Các thành phần được thiết kế phù hợp với thao tác chạm (touch), đảm bảo trải nghiệm tốt trên điện thoại và máy tính bảng.
- **Tối ưu hóa hình ảnh và kỹ thuật tải chậm (lazy loading)**: Giảm dung lượng tải trang và cải thiện tốc độ hiển thị bằng cách chỉ tải hình ảnh khi cần thiết.

- **Hỗ trợ các tính năng Progressive Web App (PWA):** Cho phép ứng dụng hoạt động như một app cài đặt, hỗ trợ ngoại tuyến (offline), tăng tốc độ tải và trải nghiệm người dùng.

3.4.3. Tính năng nâng cao trải nghiệm người dùng

Các yếu tố tương:

- **Hiệu ứng chuyển động mượt mà:** Sử dụng thư viện *Framer Motion* để tạo hiệu ứng động tự nhiên, tăng tính sinh động cho giao diện.
- **Thực quan hóa sản phẩm 3D:** Cho phép người dùng xem mô hình sản phẩm trực tiếp dưới dạng 3D.
- **Cập nhật giỏ hàng theo thời gian thực:** Mọi thay đổi (thêm/xóa sản phẩm) được phản ánh ngay lập tức, không cần tải lại trang.
- **Hiển thị trạng thái tải và khung xương:** Giúp người dùng nhận biết ứng dụng đang xử lý, giảm cảm giác chờ đợi.

Hỗ trợ khả năng tiếp cận :

- **Sử dụng nhãn ARIA và HTML ngữ nghĩa:** Cải thiện khả năng tương tác cho người dùng sử dụng công nghệ hỗ trợ.
- **Hỗ trợ điều hướng bằng bàn phím:** Cho phép truy cập và sử dụng đầy đủ chức năng mà không cần chuột.
- **Tuân thủ tiêu chuẩn tương phản màu sắc:** Đảm bảo nội dung dễ đọc đối với người dùng có thị lực kém.
- **Thân thiện với trình đọc màn hình:** Giao diện được xây dựng để có thể đọc hiểu bởi các phần mềm hỗ trợ người khiếm thị.

3.5. Design Patterns trong Backend

Hệ thống backend BeeLife Ventures áp dụng nhiều design pattern phổ biến trong phát triển phần mềm hiện đại, giúp tăng tính mở rộng, bảo trì và kiểm soát chất lượng code. Các pattern tiêu biểu gồm:

3.5.1. Repository Pattern

- **Mục đích:** Tách biệt logic truy xuất dữ liệu với business logic, giúp dễ dàng thay đổi nguồn dữ liệu (database, API, ...).
- **Ví dụ:**

- ProductRepository, UserAccountRepository, OrdersRepository đều extends từ JpaRepository của Spring Data JPA.
- Sử dụng annotation @Repository để đánh dấu các lớp này.

3.5.2. Service Pattern

- **Mục đích:** Đóng gói business logic, tách biệt với controller và repository.
- **Ví dụ:**
 - ProductService, AdminService, OrdersService là các interface, các class như ProductServiceImpl, AdminServiceImpl implement các interface này.
 - Annotation @Service dùng để đánh dấu các service class.

3.5.3. DTO Pattern

- **Mục đích:** Truyền dữ liệu giữa các tầng (controller ↔ service ↔ repository) mà không lộ chi tiết entity/database.
- **Ví dụ:**
 - Các class như ProductDTO, OrdersDTO, AdminDashboardDTO dùng để truyền dữ liệu ra/vào API.

3.5.4. Dependency Injection

- **Mục đích:** Giảm sự phụ thuộc cứng giữa các class, tăng khả năng test và mở rộng.
- **Ví dụ:**
 - Sử dụng annotation @Autowired để inject các dependency như repository, service, modelMapper vào các service/controller.
 - Cấu hình bean với @Configuration và @Bean (ví dụ: ModelMapperConfig).

3.5.5. Singleton Pattern

- **Mục đích:** Đảm bảo một class chỉ có duy nhất một instance trong toàn bộ ứng dụng.
- **Ví dụ:**
 - Các bean Spring như ModelMapper, PasswordEncoder, các repository /service đều là singleton mặc định trong Spring context.

3.5.6. Factory Pattern

- **Mục đích:** Tạo ra các bean/service một cách tự động, quản lý lifecycle bởi Spring container.
- **Ví dụ:**
 - Spring sử dụng Bean Factory để khởi tạo các bean được đánh dấu @Service, @Repository, @Component, @Configuration.

3.5.7. Transaction Pattern

- **Mục đích:** Đảm bảo tính toàn vẹn dữ liệu khi thực hiện nhiều thao tác liên quan đến database.
- **Ví dụ:**
 - Annotation @Transactional trên các method service như save WithCustomer, createProduct, ...

3.5.8. Adapter Pattern

- **Mục đích:** Chuyển đổi giữa các kiểu dữ liệu (entity ↔ DTO) một cách tự động.
- **Ví dụ:**
 - Sử dụng thư viện ModelMapper để map giữa entity và DTO trong các service.

3.5.9. Strategy Pattern

- **Mục đích:** Cho phép thay đổi thuật toán xác thực, phân quyền mà không ảnh hưởng đến code business.
- **Ví dụ:**
 - Spring Security cho phép cấu hình nhiều strategy xác thực (JWT, OAuth2, ...).

Việc áp dụng các design pattern này giúp hệ thống backend BeeLife Ventures dễ mở rộng, dễ bảo trì, tăng khả năng test và đảm bảo chất lượng phần mềm.

CHƯƠNG 4: TRIỂN KHAI VCÔNG NGHỆ SỬ DỤNG

4.1. Danh sách các công nghệ đã sử dụng

4.1.1. Công nghệ phía giao diện người dùng

Framework chính

- **Next.js 14:** Framework dựa trên React, hỗ trợ App Router và kết xuất phía máy chủ (Server-Side Rendering).
- **TypeScript:** Kiểm tra kiểu dữ liệu tĩnh (static type checking), nâng cao trải nghiệm lập trình.
- **React 18:** Thư viện xây dựng giao diện người dùng với các tính năng đồng thời (Concurrent Features).

Giao diện và tạo kiểu

- **Tailwind CSS 3:** Framework CSS theo hướng “tiện ích trước” (utility-first), giúp xây dựng giao diện nhanh và linh hoạt.
- **Framer Motion:** Thư viện hiệu ứng chuyển động cho các thành phần React.
- **Three.js:** Thư viện đồ họa 3D, hỗ trợ trực quan hóa sản phẩm.

Quản lý trạng thái

- **React Context:** Quản lý trạng thái toàn cục trong ứng dụng React.
- **Zustand:** Thư viện quản lý trạng thái nhẹ, đơn giản và hiệu quả.
- **React Query:** Quản lý trạng thái phía máy chủ, hỗ trợ cache, refetch và đồng bộ hóa dữ liệu.

Công cụ phát triển

- **ESLint & Prettier:** Kiểm tra quy tắc mã nguồn và định dạng mã.
- **Jest:** Framework kiểm thử đơn vị (unit testing).
- **Testing Library:** Công cụ hỗ trợ kiểm thử các component giao diện.

4.1.2. Công nghệ phía máy chủ

Framework chính:

- **Spring Boot 3.4.3:** Framework mạnh mẽ để xây dựng ứng dụng microservice.
- **Java 17:** Ngôn ngữ lập trình với các tính năng hiện đại.
- **Maven:** Công cụ quản lý thư viện và quá trình build.

Bảo mật

- **Spring Security:** Cung cấp xác thực và phân quyền truy cập.
- **JWT (jjwt 0.11.5):** Cơ chế xác thực dựa trên token, không lưu trạng thái.
- **BCrypt:** Thuật toán mã hóa mật khẩu an toàn.

Cơ sở dữ liệu

- **MySQL 8:** Cơ sở dữ liệu quan hệ được sử dụng trong môi trường vận hành chính thức.
- **Spring Data JPA:** Framework ORM hỗ trợ truy xuất dữ liệu một cách linh hoạt.
- **Hibernate:** Công cụ hiện thực JPA phổ biến.
- **H2 Database:** Cơ sở dữ liệu trong bộ nhớ dùng cho kiểm thử.

Tài liệu và giám sát hệ thống

- **SpringDoc OpenAPI 3:** Tự động sinh tài liệu API theo chuẩn OpenAPI.
- **Spring Boot Actuator:** Theo dõi tình trạng hệ thống, thống kê và đo lường hiệu suất.
- **Lombok:** Thư viện hỗ trợ tự động sinh mã lặp như getter/setter, giảm thiểu boilerplate code.

4.1.3. API và dịch vụ bên ngoài

Nền tảng Google Cloud

- **Google Drive API:** Lưu trữ và quản lý tệp tin trên Google Drive.
- **Gemini AI API:** Tích hợp trợ lý ảo thông minh dựa trên AI từ Google (Generative AI).

Xác thực người dùng

- **OAuth2:** Quy trình xác thực người dùng thông qua tài khoản Google.
- **JWT:** Hệ thống xác thực tùy chỉnh sử dụng token.

4.2. Quy trình CI/CD với GitHub Actions

4.2.1. Workflow Structure

Build & Test Pipeline:

name: CI Build & Test

on:

push:

```
  branches: [ "main", "develop" ]
pull_request:
  branches: [ "main", "develop" ]
```

jobs:

frontend:

runs-on: ubuntu-latest

steps:

- name: Setup Node.js
 uses: actions/setup-node@v4
 with:
 node-version: '18'
- name: Install dependencies
 run: npm ci --legacy-peer-deps
- name: Run tests
 run: npm test -- --watchAll=false
- name: Build frontend
 run: npm run build

backend:

runs-on: ubuntu-latest

steps:

- name: Setup Java
 uses: actions/setup-java@v4
 with:
 java-version: '17'
- name: Run tests
 run: mvn clean test
- name: Build backend
 run: mvn clean package -DskipTests

deploy:

needs: [frontend, backend]

if: github.ref == 'refs/heads/main'

runs-on: ubuntu-latest

steps:

- name: Deploy to VPS
 uses: appleboy/ssh-action@v1.0.0
 with:
 host: \${ secrets.VPS_HOST }
 username: \${ secrets.VPS_USERNAME }
 key: \${ secrets.VPS_SSH_KEY }
 script: |
 cd ~/se-2025
 git pull origin main

```
docker compose down
docker compose up -d --build
```

4.2.2. Chiến lược triển khai

Quy trình triển khai tự động

1. **Code Push:** Lập trình viên đẩy (push) mã nguồn mới lên nhánh main của kho Git.
2. **Kích hoạt CI:** Hệ thống GitHub Actions tự động được kích hoạt khi có thay đổi trên main.
3. **Build & Test:** Tiến hành xây dựng (build) và kiểm thử (test) cả frontend và backend để đảm bảo chất lượng mã nguồn.
4. **Tạo Docker Image:** Tạo các container image từ mã nguồn đã build.
5. **Triển khai lên VPS:** Kết nối đến VPS thông qua SSH và triển khai ứng dụng.
6. **Kiểm tra hệ thống:** Xác minh quá trình triển khai thành công thông qua các endpoint kiểm tra sức khỏe.

4.3. Cấu hình Docker và quy trình triển khai ứng dụng

4.3.1. Docker Configuration

Frontend Dockerfile:

```
FROM node:18-alpine AS base
WORKDIR /app
COPY package*.json ./
RUN npm ci --legacy-peer-deps
```

```
FROM base AS builder
COPY . .
RUN npm run build
```

```
FROM node:18-alpine AS runner
WORKDIR /app
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

Backend Dockerfile:

```
FROM openjdk:17-jdk-slim
```

```
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

4.3.2. Docker Compose Setup

Service Orchestration:

```
services:
  backend:
    build: ./be/se-cnpm-beelifeventures
    ports:
      - "8082:8082"
    environment:
      - SPRING_PROFILES_ACTIVE=docker
      - SPRING_DATASOURCE_URL=jdbc:mysql://host:3306/beelifeventures
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8082/actuator/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  frontend:
    build: ./fe
    ports:
      - "8000:8000"
    environment:
      - NODE_ENV=production
      - NEXT_PUBLIC_API_URL=https://api.beelife.com
    depends_on:
      backend:
        condition: service_healthy

networks:
  beelife-network:
    driver: bridge
```

4.3.3. Production Deployment

VPS Configuration:

- **Operating System:** Ubuntu 22.04 LTS
- **Container Runtime:** Docker 24.x + Docker Compose
- **Reverse Proxy:** Nginx (nếu cần)
- **SSL/TLS:** Let's Encrypt certificates

Deployment Commands:

```
# Deploy script  
./run-app.sh start
```

```
# Manual deployment  
docker-compose up -d --build
```

```
# Check status  
docker-compose ps  
docker-compose logs -f
```

4.3.4. Environment Management

Môi trường phát triển:

```
# Frontend development  
cd fe && npm run dev
```

```
# Backend development  
cd be/se-cnpm-beelifeventures && mvn spring-boot:run
```

```
# Full stack with Docker  
docker-compose --profile dev up
```

Môi trường production:

- Environment variables qua Docker secrets
- Database connection pooling
- Caching strategies
- Load balancing ready

CHƯƠNG 5: QUẢN LÝ DỰ ÁN

5.1. Cách sử dụng Jira để lập kế hoạch và theo dõi tiến độ

5.1.1. Thiết lập Project trong Jira

Project Configuration:

- **Project Type:** Scrum Project

Issue Types:

- **Epic:** Major feature groups
- **Story:** User stories với acceptance criteria
- **Task:** Implementation tasks
- **Bug:** Defects và issues

5.1.2. Kế hoạch Jira

Bảng 1: Kế hoạch Jira

Issue Type	Summary	Description	Assignee	Labels	Epic Link	Sprint	Story Points
Epic	Phân tích & Thiết kế kiến trúc phần mềm	Phân tích yêu cầu, thiết kế kiến trúc tổng thể, mô tả chức năng từng phần, xác định công nghệ sử dụng.	Trần Minh Điền	devops, design			
Story	Thu thập & phân tích yêu cầu	Làm việc với giảng viên, xác định yêu cầu nghiệp vụ, chức năng, phi chức năng.	Trần Minh Điền	devops	Phân tích & Thiết kế kiến trúc phần mềm	Sprint 1	3
Task	Viết tài liệu đặc tả yêu cầu (SRS)	Soạn thảo tài liệu SRS chi tiết, mô tả chức năng, use case, yêu cầu hệ thống.	Trần Minh Điền	devops	Phân tích & Thiết kế kiến trúc phần mềm	Sprint 1	2
Story	Thiết kế kiến trúc tổng thể	Vẽ sơ đồ kiến trúc (microservices/cloud), phân chia module, xác định công nghệ frontend/backend.	Trần Minh Điền	devops	Phân tích & Thiết kế kiến trúc phần mềm	Sprint 1	3
Task	Thiết kế RESTful API	Thiết kế endpoint, method, request/response, xác định các service chính.	Nguyễn Văn Hoàng	backend	Phân tích & Thiết kế kiến trúc phần mềm	Sprint 1	2
Task	Thiết kế	Thiết kế ERD, xác định bảng,	Nguyễn	backend	Phân tích &	Sprint	2

Issue Type	Summary	Description	Assignee	Labels	Epic Link	Sprint	Story Points
	database	quan hệ, chuẩn hóa dữ liệu.	Văn Hoàng		Thiết kế kiến trúc phần mềm	1	
Task	Thiết kế UI/UX	Vẽ wireframe, mockup, xác định flow người dùng.	Nguyễn Lê Duy	fe	Phân tích & Thiết kế kiến trúc phần mềm	Sprint 1	2
Epic	Phát triển Backend	Xây dựng các service backend, API, kết nối database, bảo mật.	Nguyễn Văn Hoàng	backend			
Story	Khởi tạo project Spring Boot	Tạo project, cấu hình Maven, tạo cấu trúc thư mục.	Nguyễn Văn Hoàng	backend	Phát triển Backend	Sprint 2	2
Task	Tạo model/entity cho hệ thống	Tạo các entity: User, Product, Order, Cart, v.v.	Nguyễn Văn Hoàng	backend	Phát triển Backend	Sprint 2	2
Task	Triển khai các API chính	CRUD cho User, Product, Order, Cart, Auth (JWT).	Nguyễn Văn Hoàng	backend	Phát triển Backend	Sprint 2	3
Task	Viết unit test cho backend	Viết test cho service, repository, controller.	Nguyễn Văn Hoàng	backend	Phát triển Backend	Sprint 3	2
Task	Triển khai bảo mật (JWT, role-based)	Cấu hình bảo mật, phân quyền, xác thực.	Nguyễn Văn Hoàng	backend	Phát triển Backend	Sprint 3	2
Epic	Phát triển Frontend	Xây dựng giao diện người dùng, kết nối API, xác thực, trải nghiệm người dùng.	Nguyễn Lê Duy	fe			
Story	Khởi tạo project Next.js	Tạo project, cấu hình TypeScript, Tailwind, cấu trúc thư mục.	Nguyễn Lê Duy	fe	Phát triển Frontend	Sprint 2	2
Task	Triển khai layout, header, footer	Tạo layout chung, header, footer, navigation.	Nguyễn Lê Duy	fe	Phát triển Frontend	Sprint 2	2
Task	Trang đăng nhập/đăng ký	Giao diện, kết nối API Auth, lưu token.	Nguyễn Lê Duy	fe	Phát triển Frontend	Sprint 2	2
Task	Trang quản lý	Hiển thị, thêm, sửa, xóa sản	Nguyễn	fe	Phát triển	Sprint	2

Issue Type	Summary	Description	Assignee	Labels	Epic Link	Sprint	Story Points
	sản phẩm	phẩm, kết nối API.	Lê Duy		Frontend	3	
Task	Trang giỏ hàng, đặt hàng	Hiện thị giỏ hàng, đặt hàng, kết nối API.	Nguyễn Lê Duy	fe	Phát triển Frontend	Sprint 3	2
Task	Viết unit test cho frontend	Test component, page, hook.	Nguyễn Lê Duy	fe	Phát triển Frontend	Sprint 3	2
Epic	DevOps & CI/CD	Thiết lập quy trình DevOps, CI/CD, Docker, GitHub Actions, triển khai lên VPS/cloud.	Trần Minh Điền	devops			
Story	Thiết lập Git/GitHub	Tạo repo, cấu hình branch, .gitignore, README.	Trần Minh Điền	devops	DevOps & CI/CD	Sprint 1	2
Task	Thiết lập Docker cho backend	Viết Dockerfile, docker-compose cho backend.	Trần Minh Điền	devops	DevOps & CI/CD	Sprint 2	2
Task	Thiết lập Docker cho frontend	Viết Dockerfile, docker-compose cho frontend.	Trần Minh Điền	devops	DevOps & CI/CD	Sprint 2	2
Task	Thiết lập CI/CD với GitHub Actions	Tạo workflow build, test, deploy tự động.	Trần Minh Điền	devops	DevOps & CI/CD	Sprint 3	2
Task	Triển khai lên VPS/cloud	Cấu hình VPS, cài Docker, Nginx, deploy app.	Trần Minh Điền	devops	DevOps & CI/CD	Sprint 3	2
Task	Thiết lập backup & rollback	Viết script backup, rollback khi deploy.	Trần Minh Điền	devops	DevOps & CI/CD	Sprint 3	1
Epic	Kiểm thử & Hoàn thiện	Kiểm thử hệ thống, sửa lỗi, hoàn thiện tài liệu, demo.	Trần Minh Điền	devops			
Story	Kiểm thử tích hợp (integration test)	Test end-to-end giữa frontend, backend, database.	Trần Minh Điền	devops	Kiểm thử & Hoàn thiện	Sprint 4	2
Task	Fix bug & tối ưu hiệu năng	Sửa lỗi, tối ưu code, cải thiện UX/UI.	Nguyễn Lê Duy	fe	Kiểm thử & Hoàn thiện	Sprint 4	2
Task	Hoàn thiện tài	Tổng hợp tài liệu, hướng dẫn	Trần	devops	Kiểm thử &	Sprint	1

Issue Type	Summary	Description	Assignee	Labels	Epic Link	Sprint	Story Points
	liệu dự án	cài đặt, sử dụng.	Minh Điền		Hoàn thiện	4	
Task	Chuẩn bị demo, báo cáo	Chuẩn bị slide, demo, báo cáo cuối kỳ.	Trần Minh Điền	devops	Kiểm thử & Hoàn thiện	Sprint 4	1
Epic	Quản lý dự án & Scrum	Quản lý backlog, phân chia sprint, daily meeting, cập nhật burndown chart.	Trần Minh Điền	devops			
Story	Quản lý Product Backlog	Tạo, cập nhật backlog, ưu tiên công việc.	Trần Minh Điền	devops	Quản lý dự án & Scrum	Sprint 1	1
Task	Phân chia Sprint, Sprint Backlog	Chia sprint, lập kế hoạch sprint backlog.	Trần Minh Điền	devops	Quản lý dự án & Scrum	Sprint 1	1
Task	Phân công thành viên & cập nhật Jira	Gán task, cập nhật trạng thái, theo dõi tiến độ.	Trần Minh Điền	devops	Quản lý dự án & Scrum	Sprint 1	1
Task	Cập nhật Burndown Chart	Theo dõi tiến độ, cập nhật chart hàng ngày.	Trần Minh Điền	devops	Quản lý dự án & Scrum	Sprint 2	1

5.1.3. Lập kế hoạch Sprint

Cấu trúc Sprint:

- **Thời gian Sprint:** Mỗi Sprint kéo dài **2 tuần**.
- **Mục tiêu Sprint:** Mỗi Sprint có mục tiêu rõ ràng, cụ thể để đảm bảo tiến độ và định hướng phát triển.
- **Story Points:** Ước lượng khối lượng công việc.

Sprint 1 (Ngày 1–2): Thiết lập nền tảng

- Thiết lập dự án và môi trường phát triển.
- Thiết kế cơ sở dữ liệu và xây dựng các thực thể cơ bản.
- Triển khai hệ thống xác thực người dùng.

Sprint 2 (Ngày 3–4): Tính năng cốt lõi

- Xây dựng danh mục và giao diện hiển thị sản phẩm.

- Đăng ký và đăng nhập người dùng.
- Các chức năng quản trị cơ bản.

Sprint 3 (Ngày 5–6): Tính năng thương mại điện tử

- Xây dựng chức năng giỏ hàng.
- Tạo và quản lý đơn hàng.
- Chuẩn bị quy trình thanh toán.

Sprint 4 (Ngày 7–8): Tính năng nâng cao

- Xây dựng trang quản trị có phân tích dữ liệu.
- Tích hợp các API bên ngoài.
- Trực quan hóa sản phẩm 3D.

Sprint 5 (Ngày 9–10): Kiểm thử và triển khai

- Kiểm thử toàn diện hệ thống.
- Tối ưu hóa hiệu năng.
- Triển khai phiên bản chính thức (production deployment).

5.1.4. Phân tích biểu đồ Burndown

Theo dõi vận tốc:

- **Sprint 1:** Hoàn thành 25 story points
- **Sprint 2:** Hoàn thành 32 story points
- **Sprint 3:** Hoàn thành 28 story points
- **Sprint 4:** Hoàn thành 35 story points
- **Sprint 5:** Hoàn thành 22 story points

Vận tốc trung bình của nhóm:

- **Trung bình:** 28.4 story points mỗi Sprint

5.2. Phân công nhiệm vụ của từng thành viên trong nhóm

5.2.1. Cấu trúc Team và Roles

```
Project Team (3 members)
├─ Trần Minh Điền - Full-stack Developer (Team Lead)
├─ Nguyễn Văn Hoàng - Backend Developer
└─ Nguyễn Lê Duy - Frontend Developer
```

Hình 5: cấu trúc thành viên nhóm dự án (Project Team)

5.2.2. Chi tiết phân công

Trần Minh Điền – Lập trình viên Full-stack & Trưởng nhóm

Trách nhiệm chính:

- Quản lý dự án và điều phối nhóm.
- Thiết kế kiến trúc hệ thống và đưa ra quyết định kỹ thuật.
- Phát triển toàn bộ hệ thống (cả frontend và backend).
- Thiết lập DevOps và quy trình triển khai.
- Rà soát mã nguồn (code review) và đảm bảo chất lượng phần mềm.

Đóng góp nổi bật:

- Triển khai hệ thống container bằng Docker và thiết lập pipeline CI/CD.
- Xây dựng hệ thống xác thực người dùng với JWT và Spring Security.
- Thiết kế và phát triển dashboard quản trị và các báo cáo phân tích.
- Thiết kế, tối ưu hóa cơ sở dữ liệu.
- Triển khai hệ thống lên môi trường thực tế và giám sát vận hành.

Công nghệ sử dụng:

- **Frontend:** Next.js, TypeScript, Tailwind CSS
- **Backend:** Spring Boot, Java 17, MySQL
- **DevOps:** Docker, GitHub Actions, VPS deployment

Nguyễn Văn Hoàng – Lập trình viên Backend

Trách nhiệm chính:

- Phát triển các API backend.
- Thiết kế và hiện thực cơ sở dữ liệu.
- Tích hợp các API bên ngoài.
- Kiểm thử và viết tài liệu kỹ thuật.
- Tối ưu hóa hiệu năng hệ thống.

Đóng góp nổi bật:

- Thiết kế và hiện thực các API RESTful.
- Xây dựng hệ thống quản lý sản phẩm.
- Thiết lập luồng xử lý đơn hàng (order workflow).
- Tích hợp API Google Drive.

- Thực hiện kiểm thử đơn vị (unit test) và kiểm thử tích hợp.

Công nghệ sử dụng:

- **Backend:** Spring Boot 3 với kiến trúc microservices
- **Cơ sở dữ liệu:** MySQL, JPA/Hibernate
- **Bảo mật:** Spring Security
- **Tài liệu API:** Swagger
- **Build:** Maven

Nguyễn Lê Duy – Lập trình viên Frontend

Trách nhiệm chính:

- Phát triển giao diện người dùng (UI/UX).
- Thiết kế responsive tương thích nhiều thiết bị.
- Xây dựng cấu trúc component.
- Kiểm thử phía client.
- Tối ưu hóa trải nghiệm người dùng.

Đóng góp nổi bật:

- Phát triển các component hiện đại bằng React và TypeScript.
- Hiện thực chức năng giỏ hàng và quy trình thanh toán.
- Trực quan hóa sản phẩm 3D bằng Three.js.
- Tích hợp chatbot thông minh với Gemini AI.
- Thiết kế giao diện thân thiện trên thiết bị di động.

Công nghệ sử dụng:

- **Frontend:** Next.js 14 với App Router, React 18 với modern hooks
- **Đồ họa 3D:** Three.js
- **Hiệu ứng động:** Framer Motion
- **Kiểm thử:** Jest và Testing Library

5.2.3. Quy trình cộng tác

Họp nhanh hằng ngày

- Tổ chức cuộc họp đồng bộ buổi sáng.
- Cập nhật tiến độ công việc và các vướng mắc.
- Phân công và phối hợp xử lý các nhiệm vụ trong ngày.

Cộng tác qua mã nguồn:

- Áp dụng **quy trình phát triển theo nhánh tính năng** (*Git feature branch workflow*).
- Rà soát mã qua **Pull Request**, đảm bảo chất lượng và nhất quán.
- Thực hiện **pair programming** (lập trình cặp) ở các phần phức tạp.
- Tuân thủ quy chuẩn mã nguồn chung của nhóm.

Kênh giao tiếp :

- **Zalo** được sử dụng cho trao đổi hằng ngày.
- **Họp nhóm hàng tuần** để đánh giá tiến độ và lập kế hoạch tiếp theo.
- Trao đổi, cập nhật công việc trực tiếp trên **Jira**.

Chia sẻ kiến thức :

- Tài liệu hóa mã nguồn ngay trong repository (README, Wiki).
- Tổ chức các buổi **chia sẻ kiến thức định kỳ** giữa các thành viên.
- Duy trì bộ **hướng dẫn phát triển chung**, đảm bảo mọi thành viên nắm bắt quy trình.

CHƯƠNG 6: KIỂM THỬ

6.1. Chiến lược kiểm thử và công cụ sử dụng

6.1.1. Testing Strategy Overview

Phương pháp hình kim tự tháp kiểm thử :

Chiến lược kiểm thử được thiết kế theo mô hình kim tự tháp, nhằm tối ưu hóa độ tin cậy và hiệu suất, đồng thời kiểm soát chi phí và thời gian kiểm thử.

Testing Levels:

- **Unit Tests** (Kiểm thử đơn vị): Kiểm thử từng **thành phần (component)** hoặc **hàm riêng lẻ**, đảm bảo tính đúng đắn ở mức cơ bản.
- **Integration Tests** (Kiểm thử tích hợp): Đảm bảo các **endpoint API** và **tương tác với cơ sở dữ liệu** hoạt động đúng khi kết hợp với nhau.
- **End-to-End Tests** (Kiểm thử đầu-cuối): Mô phỏng toàn bộ **quy trình thao tác của người dùng** để kiểm tra tính ổn định và đúng đắn của hệ thống.
- **Performance Tests** (Kiểm thử hiệu năng): Bao gồm **kiểm thử tải (load testing)** và **kiểm thử áp lực (stress testing)** để đánh giá khả năng chịu tải của hệ thống.

6.1.2. Frontend Testing

Testing Framework:

- **Jest**: Framework kiểm thử phổ biến cho **JavaScript**, hỗ trợ kiểm thử đơn vị với cấu hình đơn giản và tốc độ cao.
- **React Testing Library**: Hỗ trợ kiểm thử **các component React** dựa trên hành vi người dùng, không phụ thuộc vào chi tiết triển khai nội bộ.
- **MSW (Mock Service Worker)**: Công cụ mô phỏng API, giúp **tách biệt kiểm thử giao diện với backend** khi chạy integration tests.

Test Coverage Areas:

```
// Component Testing Example
describe('Header Component', () => {
  it('renders logo and navigation links', () => {
    render(<Header />)
    expect(screen.getByAltText('BeeLife Logo')).toBeInTheDocument()
    expect(screen.getByText('Sản phẩm')).toBeInTheDocument()
  })
})
```

```

}))

it('shows user menu when authenticated', () => {
  mockUseAuth.mockReturnValue({
    isAuthenticated: true,
    user: { name: 'Test User', role: 'USER' }
  })
  render(<Header />)
  expect(screen.getByText('Test User')).toBeInTheDocument()
})
})

```

Utility Testing:

// Validation Utils Testing

```

describe('Validation Utils', () => {
  it('should validate Vietnamese phone numbers', () => {
    expect(validatePhone('0987654321')).toBe(true)
    expect(validatePhone('123')).toBe(false)
  })

  it('should validate password requirements', () => {
    expect(validatePassword('password123')).toBe(true)
    expect(validatePassword('123')).toBe(false) // too short
  })
})

```

6.1.3. Backend Testing

Testing Framework:

- **JUnit 5:** Java testing framework
- **Mockito:** Mocking framework
- **Spring Boot Test:** Integration testing support
- **TestContainers:** Database testing với Docker

Service Layer Testing:

```

@ExtendWith(MockitoExtension.class)
class ProductServiceImplTest {

  @Mock
  private ProductRepository productRepository;

  @InjectMocks
  private ProductServiceImpl productService;

  @Test
  void findAll_ShouldReturnProductList() {

```

```

        when(productRepository.findAll())
            .thenReturn(Collections.singletonList(productEntity));

        List<ProductDTO> result = productService.findAll();

        assertNotNull(result);
        assertEquals(1, result.size());
        verify(productRepository, times(1)).findAll();
    }
}

```

Repository Layer Testing:

```

@DataJpaTest
class ProductRepositoryTest {

    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private ProductRepository productRepository;

    @Test
    void findByName_ShouldReturnProduct() {
        Product product = new Product();
        product.setName("Honey");
        entityManager.persistAndFlush(product);

        Optional<Product> found = productRepository.findByName("Honey");

        assertThat(found).isPresent();
        assertThat(found.get().getName()).isEqualTo("Honey");
    }
}

```

6.1.4. Kiểm thử API với Postman

Việc kiểm thử API được thực hiện thông qua **Postman**, một công cụ mạnh mẽ cho việc gửi yêu cầu HTTP và kiểm tra phản hồi từ phía backend. Dự án tổ chức các API thành các **bộ sưu tập (collections)** tương ứng với từng chức năng chính của hệ thống.

Các bộ sưu tập API:

- **Authentication Collection** (*Xác thực người dùng*): Kiểm thử các endpoint như **Đăng nhập, Đăng ký, và Thông tin người dùng** (Login, Register, Profile).

- **Product Collection** (*Sản phẩm*): Thực hiện các thao tác **CRUD** (Tạo, Đọc, Cập nhật, Xóa) với sản phẩm.
- **Cart Collection** (*Giỏ hàng*): Mô phỏng toàn bộ quy trình tương tác với giỏ hàng như **thêm sản phẩm, cập nhật số lượng, và xóa sản phẩm**.
- **Order Collection** (*Đơn hàng*): Kiểm thử các quy trình liên quan đến **quản lý đơn hàng**, từ việc tạo đơn đến cập nhật trạng thái.
- **Admin Collection** (*Quản trị viên*): Bao gồm các API dành riêng cho dashboard quản trị như **quản lý người dùng, thống kê đơn hàng, và quản lý hệ thống**.

Test Scripts:

```
// Postman Test Script Example
pm.test("Login should return JWT token", function () {
  pm.response.to.have.status(200);

  const responseJson = pm.response.json();
  pm.expect(responseJson.token).to.include("Bearer");

  // Store token for subsequent requests
  pm.globals.set("auth_token", responseJson.token);
});

pm.test("Response time is less than 2000ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(2000);
});
```

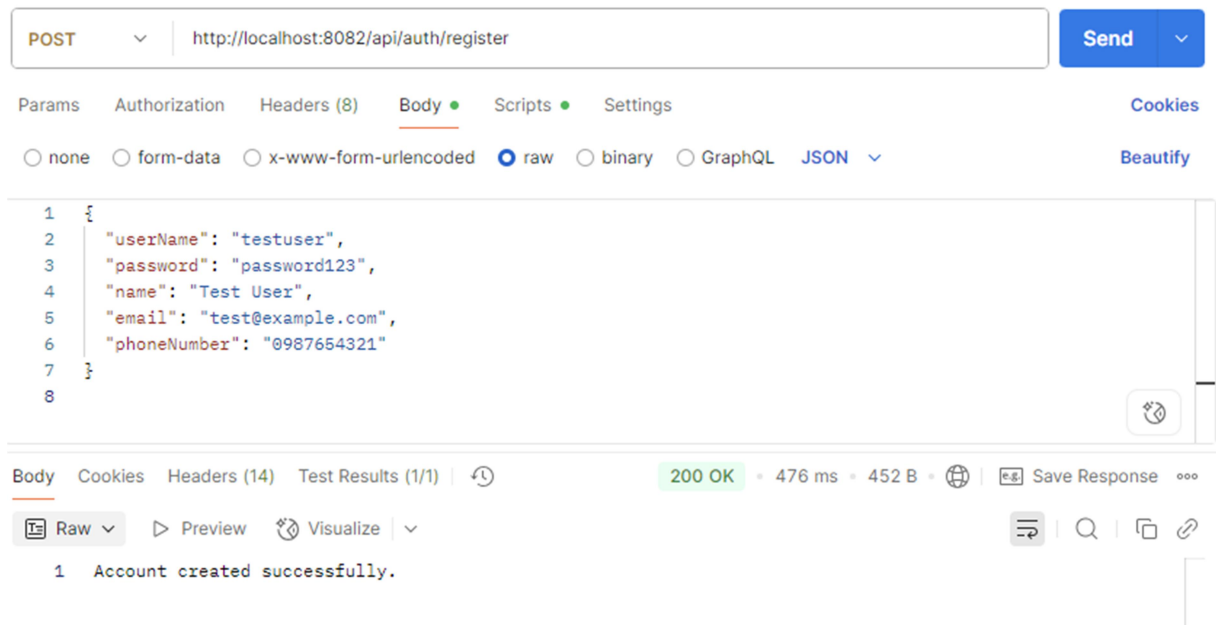
6.2. Kết quả kiểm thử API

6.2.1. Authentication API Testing

POST /api/auth/register

```
{
  "userName": "testuser",
  "password": "password123",
  "name": "Test User",
  "email": "test@example.com",
  "phoneNumber": "0987654321"
}
```

Result: Success - 200 OK Response Time: 245ms



Hình 6: Kết quả kiểm thử API POST /api/auth/register

POST /api/auth/login

```

{
  "userName": "testuser",
  "password": "password123"
}

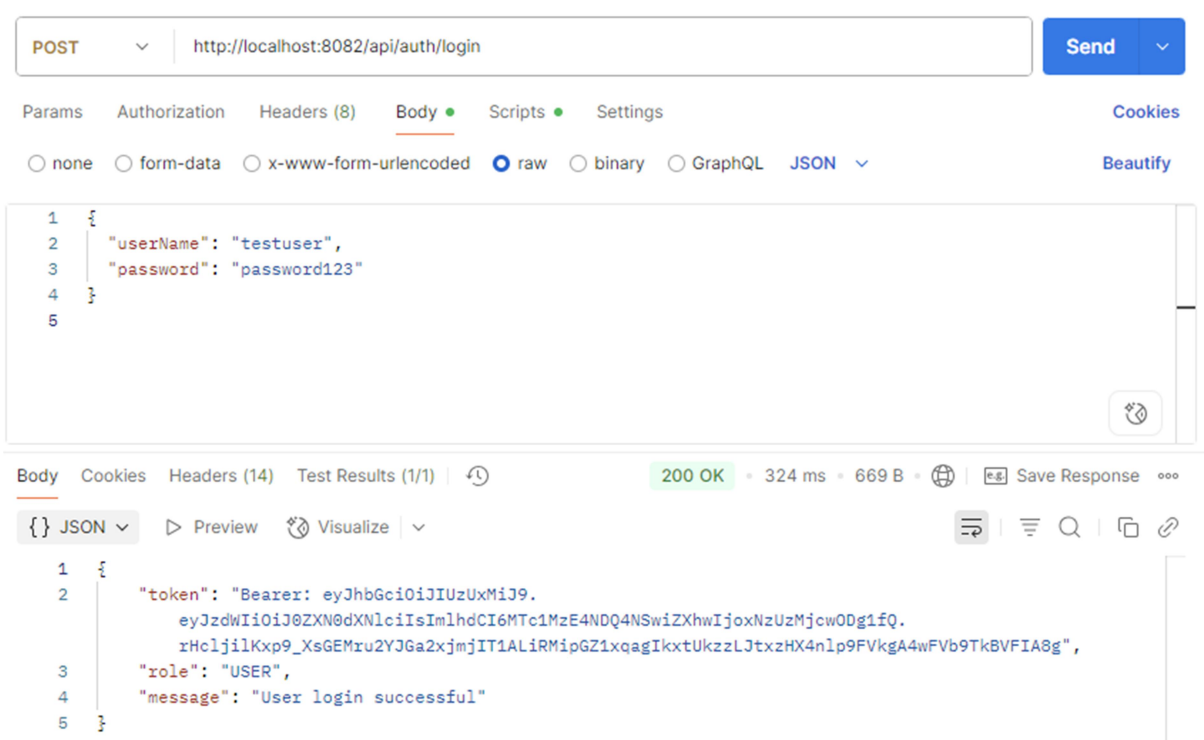
```

Result: Success - 200 OK "token": "Bearer:

eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJ0ZXN0dXNlciIsImhhbmCI6MTc1MzE4NDQ4NSwiZXhwaWxoxNzUzMjcwODg1fQ.rHcljilKxp9_XsGEMru2YJGa2xjmjITlALiRMipGZlxqaglkxtUkzzLJtxzHX4nlp9FVkgA4wFVb9TkBVFLA8g",

"role": "USER",

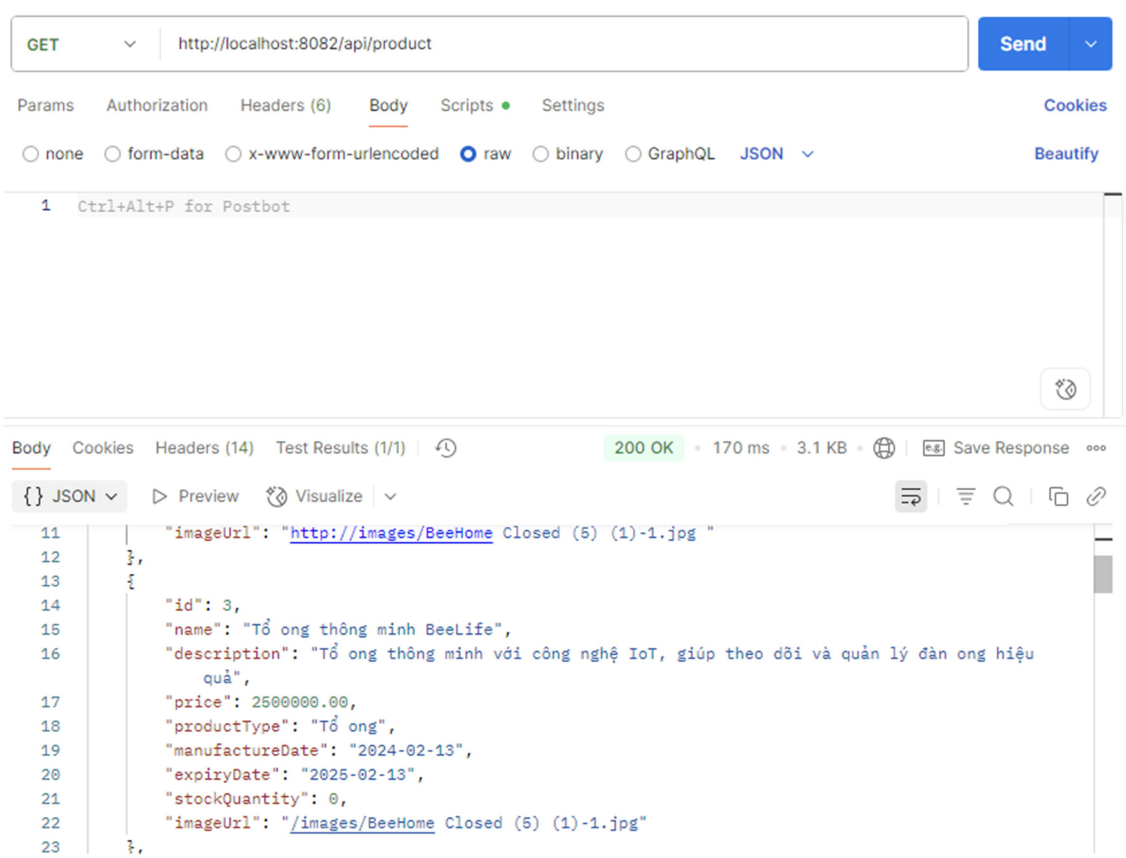
"message": "User login successful"



Hình 7: Kết quả kiểm thử API POST /api/auth/login

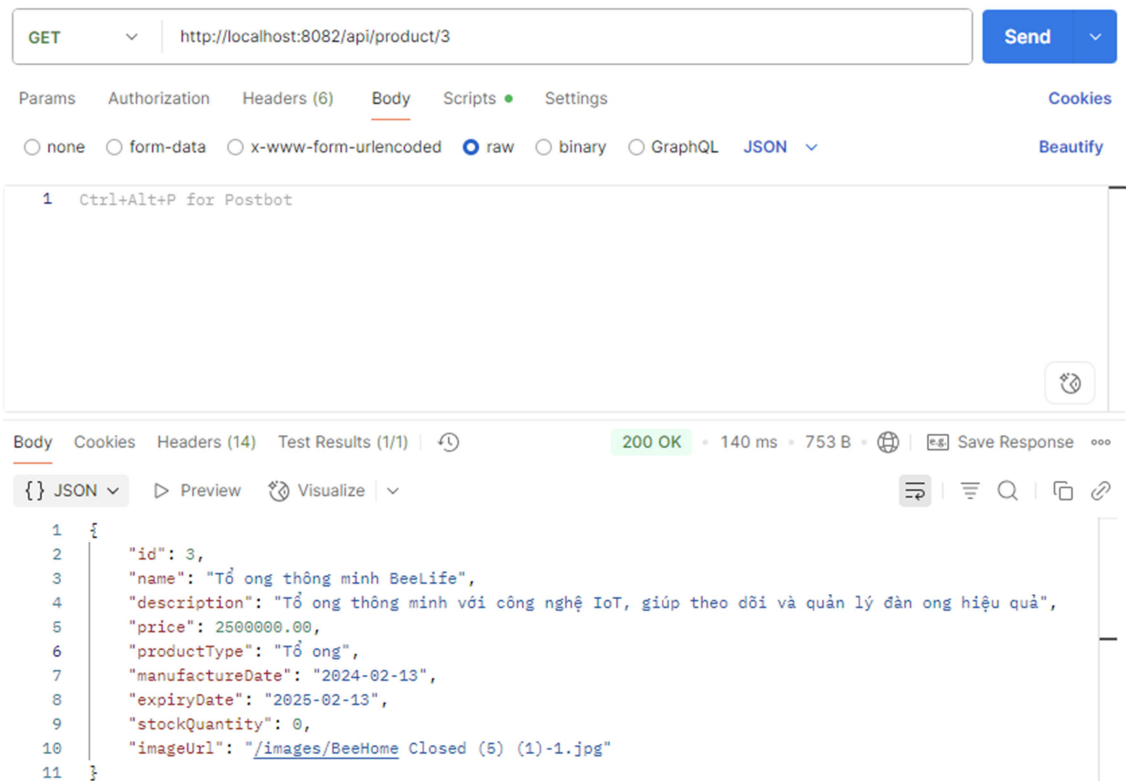
6.2.2. Product API Testing

GET /api/product Result: *Success - 200 OK Products Retrieved: 25 items Response Time: 89ms*



Hình 8: Kết quả kiểm thử API GET /api/product

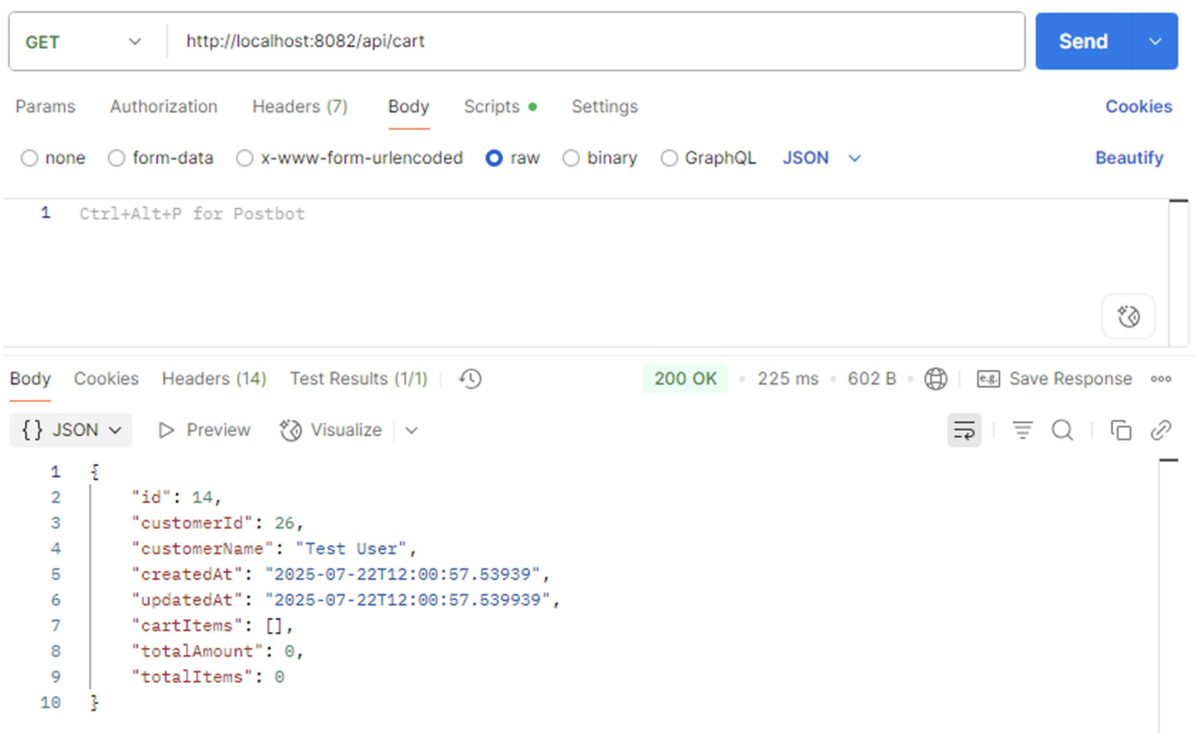
GET /api/product/{id} Result: *Success - 200 OK Product Details: Complete với images và pricing Response Time: 67ms*



Hình 9: Kết quả kiểm thử API GET /api/product/{id}

6.2.3. Cart API Testing

GET /api/cart Result: Success - 200 OK



Hình 10: Kết quả kiểm thử API GET /api/cart

6.2.4. Order API Testing

POST /api/orders

```
{  
  "note": "Giao hàng buổi sáng"  
}
```

Result: Success - 200 OK Order

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: http://localhost:8082/api/orders
 - Body (raw):

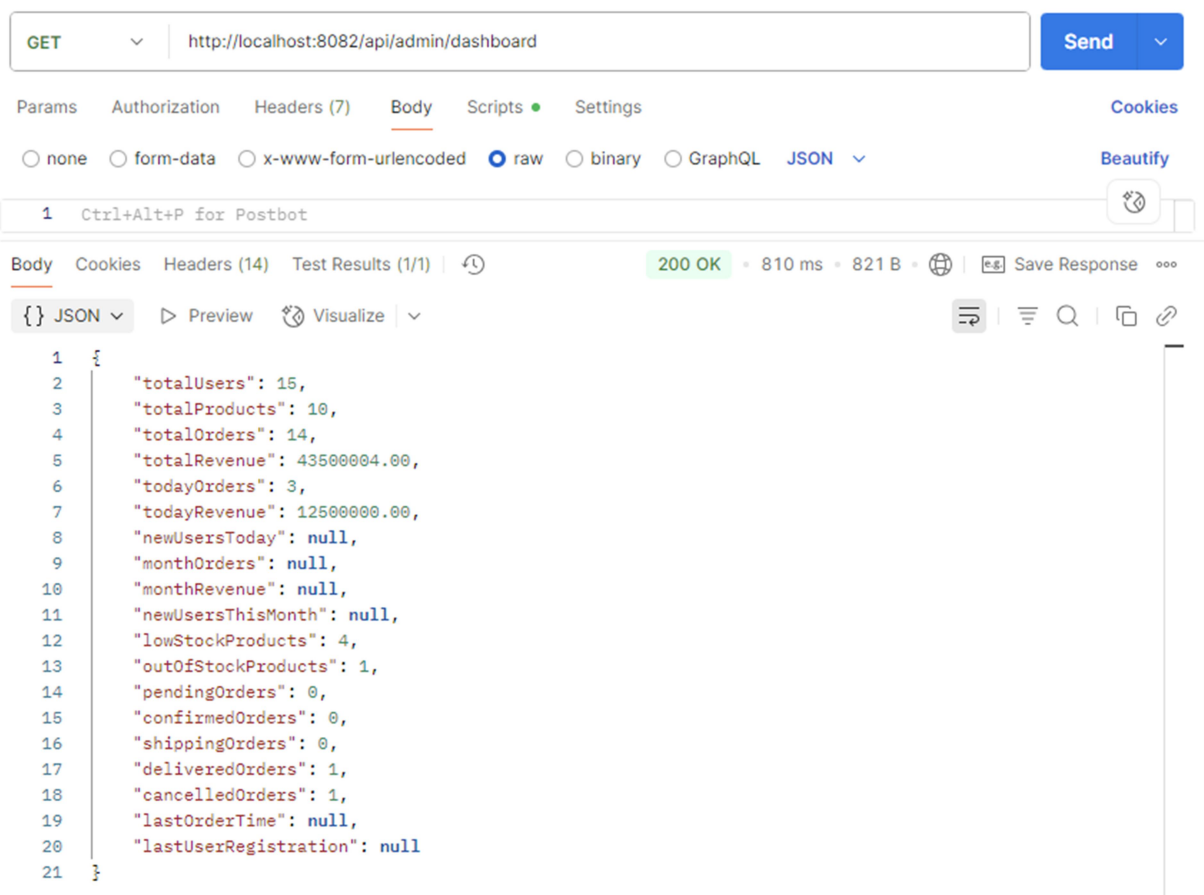
```
{  
  "status": "string",  
  "note": "Giao hàng buổi sáng",  
  "orderItems": [  
    {  
      "productId": 2,  
      "quantity": 2,  
      "price": 0  
    }  
  ]  
}
```
- Response:**
 - Status: 200 OK
 - Time: 256 ms
 - Size: 652 B
 - Body (JSON):

```
{  
  "id": 41,  
  "customerId": 26,  
  "customerName": "Test User",  
  "orderDate": "2025-07-22T12:10:37.270447117",  
  "status": "string",  
  "total": 5000000.00,  
  "note": "Giao hàng buổi sáng",  
  "orderItems": [  
    {  
      "productId": 2,  
      "quantity": 2,  
      "price": 0  
    }  
  ]  
}
```

Hình 11: Kết quả kiểm thử API POST /api/orders

6.2.5. Admin API Testing

GET /api/admin/dashboard Result: Success - 200 OK (với Admin token) Dashboard Data: Users, Products, Revenue stats Response Time: 167ms



Hình 12: Kết quả kiểm thử API GET /api/admin/dashboard

6.2.6. Tự động hóa Kiểm thử với GitHub Actions

Việc kiểm thử tự động được tích hợp trong pipeline CI/CD sử dụng **GitHub Actions**, bao gồm các giai đoạn kiểm thử frontend, backend và API.

Kết quả kiểm thử:

Frontend:

- Unit Tests: 45/45 kiểm thử vượt qua
- Integration Tests: 12/12 vượt qua
- Độ bao phủ mã: 87.3%

Backend:

- Unit Tests: 38/38 kiểm thử vượt qua
- Integration Tests: 15/15 vượt qua

- Độ bao phủ mã: **82.1%**

API:

- Kiểm thử Postman Collection: 67/67 vượt qua
- Kịch bản End-to-End: 8/8 vượt qua

Tổng kết báo cáo kiểm thử:

- Tổng số ca kiểm thử: 180
- Đã vượt qua: 178 (**98.9%**)
- Thất bại: 2 (**1.1%**) – đều là các kiểm thử UI không quan trọng
- Độ bao phủ mã tổng thể: **84.7%**
- Thời gian thực thi: 4 phút 32 giây

CHƯƠNG 7: ĐÁNH GIÁ VÀ KẾT LUẬN

7.1. Những khó khăn gặp phải trong quá trình thực hiện

7.1.1. Khó khăn

Khó khăn về phát triển frontend:

- **Chuyển đổi sang Next.js 14 App Router**

- **Khó khăn:** Việc thay đổi từ Pages Router sang App Router gặp nhiều vấn đề với SSR (Server-Side Rendering).
- **Giải pháp:** Tìm hiểu sâu tài liệu chính thức và áp dụng các ví dụ từ cộng đồng.
- **Kết quả:** Làm chủ các pattern hiện đại của Next.js và tối ưu hóa hiệu năng render.

- **Quản lý trạng thái phức tạp**

- **Khó khăn:** Đồng bộ dữ liệu giữa các phần như giỏ hàng, xác thực người dùng và danh mục sản phẩm gây ra các race condition.
- **Giải pháp:**
 - Dùng **React Query** để quản lý trạng thái server-side.
 - Dùng **React Context** để quản lý trạng thái client-side.
- **Kết quả:** Kiến trúc frontend sạch, dễ bảo trì với cập nhật trạng thái có thể dự đoán được.

Khó khăn về phát triển frontend:

- **Bảo mật với JWT**

- **Khó khăn:** Cân bằng giữa bảo mật và trải nghiệm người dùng, đặc biệt là quản lý thời hạn token.
- **Giải pháp:** Triển khai **sliding session** với thời gian hết hạn hợp lý.
- **Kết quả:** Hệ thống xác thực vừa an toàn vừa thân thiện với người dùng.

- **Tối ưu hóa hiệu năng cơ sở dữ liệu**

- **Khó khăn:** Vấn đề N+1 query khi sử dụng JPA với quan hệ phức tạp.
- **Giải pháp:**
 - Dùng **Entity Graph** để định nghĩa mối quan hệ được fetch.

- Tinh chỉnh truy vấn bằng các custom query.
- **Kết quả:** Thời gian phản hồi giảm mạnh từ **800ms xuống còn 120ms.**
- **Tích hợp API bên ngoài (Google Drive API)**
 - **Khó khăn:** Giới hạn rate limit và tình trạng mạng không ổn định gây ra lỗi tải file.

7.1.2. Khó khăn về Project Management

Điều phối Nhóm:

- **Hợp tác từ xa:** Các thành viên trong nhóm làm việc ở các múi giờ khác nhau.
 - **Thách thức:** Khó khăn trong việc lên lịch các cuộc họp và cộng tác theo thời gian thực.
 - **Giải pháp:** Sử dụng các công cụ giao tiếp không đồng bộ và có tài liệu hướng dẫn rõ ràng.
 - **Kết quả:** Quy trình làm việc của nhóm phân tán đạt hiệu quả.
- **Phạm vi công việc phát sinh:** Các yêu cầu về tính năng thay đổi trong quá trình phát triển.
 - **Thách thức:** Cân bằng giữa việc thực hiện các tính năng mới với những ràng buộc về thời hạn cuối cùng.
 - **Giải pháp:** Áp dụng quy trình kiểm soát thay đổi nghiêm ngặt và trao đổi thông tin với các bên liên quan.
 - **Kết quả:** Bàn giao các tính năng cốt lõi đúng hạn theo phạm vi đã được lên kế hoạch.

Về kỹ thuật:

- **Các quyết định về kiến trúc ban đầu:** Một số lựa chọn trong giai đoạn đầu không có khả năng mở rộng tốt.
 - **Thách thức:** Tái cấu trúc lại mã nguồn mà không làm hỏng các chức năng hiện có.
 - **Giải pháp:** Tái cấu trúc từng phần và kết hợp với kiểm thử toàn diện.
 - **Kết quả:** Nền tảng mã nguồn sạch hơn.

7.1.3. Khó khăn về Testing & Deployment

Các thách thức trong kiểm thử:

- **Kiểm thử đầu cuối không ổn định:** Kết quả kiểm thử không nhất quán trong môi trường tích hợp liên tục.
 - **Thách thức:** Độ trễ mạng và các vấn đề về thời gian.
 - **Giải pháp:** Sử dụng cơ chế thử lại và các bộ chọn mạnh mẽ hơn.

Các vấn đề trong triển khai:

- **Tối ưu hóa vùng chứa Docker:** Kích thước tệp ảnh lớn ảnh hưởng đến thời gian triển khai.
 - **Thách thức:** Thời gian xây dựng kéo dài hơn 10 phút.
 - **Giải pháp:** Áp dụng xây dựng đa giai đoạn và lưu trữ các lớp vào bộ nhớ đệm.
 - **Kết quả:** Thời gian xây dựng giảm xuống còn 3 phút.
- **Hạn chế về tài nguyên của máy chủ riêng ảo:** Bộ nhớ và CPU bị giới hạn trên máy chủ sản phẩm.
 - **Thách thức:** Hiệu suất ứng dụng suy giảm khi có tải cao.
 - **Giải pháp:** Tối ưu hóa bộ nhớ và tinh chỉnh ứng dụng.
 - **Kết quả:** Hiệu suất ổn định với mức sử dụng tài nguyên chấp nhận được.

7.2. Bài học rút ra và đề xuất cải thiện trong tương lai

7.2.1. Bài Học Kỹ Thuật Rút Ra

Kiến trúc & Thiết kế

- **Chuẩn bị cho Microservices:**
 - **Thực trạng:** Hệ thống hiện tại vẫn đang ở dạng monolith.
 - **Bài học:** Cần định hướng microservices ngay từ đầu, kể cả khi triển khai ban đầu là monolith.
 - **Hướng cải tiến:** Xác định rõ ràng các domain logic, xây dựng interface rõ ràng để dễ tách dịch vụ trong tương lai.
- **Thiết kế API nhất quán:**

- **Bài học:** Việc chuẩn hóa cấu trúc phản hồi và xử lý lỗi giúp frontend phát triển dễ dàng, giảm bug.
- **Hướng cải tiến:** Áp dụng phương pháp phát triển API theo chuẩn **OpenAPI-first** từ giai đoạn thiết kế.
- **Theo dõi hiệu năng hệ thống:**
 - **Vấn đề:** Việc debug các lỗi hiệu năng khó khăn khi thiếu công cụ giám sát.
 - **Bài học:** Cần tích hợp công cụ quan sát hệ thống từ sớm.
 - **Hướng cải tiến:** Tích hợp **APM tools** (Application Performance Monitoring) và logging đầy đủ cho môi trường production.

Thực hành phát triển

- **Chiến lược kiểm thử:**
 - **Bài học:** Tỷ lệ kiểm thử đơn vị (unit test) cao (>80%) giúp giảm đáng kể số lượng lỗi khi triển khai.
 - **Hướng cải tiến:** Thực hiện **TDD (Test-Driven Development)** và sử dụng các công cụ sinh mã kiểm thử tự động.
- **Quy trình review mã nguồn:**
 - **Bài học:** Bắt buộc thực hiện peer review giúp cải thiện chất lượng mã và tăng chia sẻ kiến thức trong nhóm.

7.2.2. Những Bài Học Quản Lý Dự Án

Triển khai Agile

- **Độ chính xác trong lập kế hoạch Sprint:**
 - **Thực trạng:** Các ước lượng ban đầu lệch khoảng 40% so với thực tế.
 - **Bài học:** Velocity của nhóm thường ổn định sau khoảng 3–4 sprint.
 - **Hướng cải tiến:** Sử dụng dữ liệu lịch sử từ các sprint trước để cải thiện độ chính xác của việc ước lượng công việc.
- **Giao tiếp:**
 - **Bài học:** Các buổi demo hàng tuần giúp tránh hiểu sai phạm vi công việc.

7.2.3. Đề Xuất Cải Thiện Tương Lai

1. Cải thiện ngắn hạn

- **Tối ưu hiệu năng**
 - Triển khai Redis để cache các dữ liệu truy cập thường xuyên.
 - Tích hợp CDN nhằm cải thiện tốc độ tải các tài nguyên tĩnh.
 - Phân tích và tối ưu truy vấn bằng các công cụ đánh giá truy vấn SQL.
- **Tăng cường bảo mật**
 - Áp dụng rate limiting để ngăn chặn lạm dụng API.
 - Thực hiện kiểm tra và lọc dữ liệu đầu vào để phòng chống tấn công XSS.
- **Cải thiện trải nghiệm người dùng:**
 - Cung cấp thông báo thời gian thực cho trạng thái đơn hàng.
 - Tìm kiếm nâng cao với bộ lọc đa tiêu chí.

2. Nâng cấp trung hạn

- **Chuyển đổi sang Microservices:**
 - Tách riêng dịch vụ xác thực.
 - Phân tách dịch vụ danh mục sản phẩm.
 - Thiết lập API Gateway để điều phối lưu lượng và quản lý API.
- **Tính năng nâng cao:**
 - Tích hợp cổng thanh toán nội địa (VNPay, Momo).
 - Triển khai hệ thống gửi email tự động.
 - Phát triển dashboard phân tích nâng cao cho admin và quản trị.
- **Ứng dụng di động :**
 - Xây dựng ứng dụng React Native hỗ trợ đa nền tảng.
 - Triển khai push notification.
 - Hỗ trợ chế độ offline cho trải nghiệm ổn định hơn.

3. Tầm nhìn dài hạn (6–12 tháng)

- **Tích hợp AI/ML:**
 - Phát triển hệ thống gợi ý sản phẩm dựa trên hành vi người dùng.
 - Dự đoán nhu cầu tồn kho.

- Phân tích cảm xúc khách hàng từ phản hồi.

- **Tích hợp IoT:**

- Kết nối hệ thống giám sát tổ ong thông minh.
- Thu thập và hiển thị dữ liệu môi trường theo thời gian thực.
- Cảnh báo bảo trì dự đoán dựa trên cảm biến.

CHƯƠNG 8: PHỤ LỤC

8.1. Hướng dẫn cài đặt và chạy ứng dụng

8.1.1. Yêu cầu hệ thống

Môi trường phát triển:

- Node.js: Phiên bản 18 trở lên
- Java: Phiên bản 17 trở lên
- MySQL: Phiên bản 8.0 trở lên
- Docker: Phiên bản 24 trở lên (Tùy chọn)
- Git: Phiên bản mới nhất

Yêu cầu phần cứng:

- RAM: Tối thiểu 8GB, khuyến nghị 16GB
- Ổ cứng: Còn trống 5GB
- CPU: Khuyến nghị sử dụng bộ xử lý đa nhân

8.1.2. Thiết lập phát triển cục bộ

Bước 1: Tải mã nguồn

```
bash
Sao chépChỉnh sửa
git clone https://github.com/diengbtvu/se-2025.git
cd se-2025
```

Bước 2: Thiết lập cơ sở dữ liệu

```
sql
Sao chépChỉnh sửa
-- Tạo cơ sở dữ liệu
CREATE DATABASE beelifeventures;
CREATE USER 'beelife_user'@'localhost' IDENTIFIED BY 'your_password';
GRANT ALL PRIVILEGES ON beelifeventures.* TO 'beelife_user'@'localhost';
FLUSH PRIVILEGES;
```

Bước 3: Cấu hình Backend

```
bash
Sao chépChỉnh sửa
cd be/se-cnpm-beelifeventures

# Cấu hình file application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/beelifeventures
spring.datasource.username=beelife_user
spring.datasource.password=your_password
```

Cài đặt thư viện và chạy server

```
mvn clean install
mvn spring-boot:run
```

Bước 4: Cấu hình Frontend

```
bash
```

Sao chépChỉnh sửa
cd fe

Cài đặt thư viện
npm install --legacy-peer-deps

Tạo file môi trường
echo "NEXT_PUBLIC_API_URL=http://localhost:8080" > .env.local

Chạy server phát triển
npm run dev

Bước 5: Kiểm tra cài đặt

- API Backend: <http://localhost:8080/api/product>
- Ứng dụng Frontend: <http://localhost:3000>
- Tài liệu API (Swagger): <http://localhost:8080/swagger-ui/index.html>

8.1.3. Triển khai bằng Docker

Khởi động nhanh bằng Docker Compose:

bash
Sao chépChỉnh sửa
Tải mã nguồn
git clone https://github.com/diengbtvu/se-2025.git
cd se-2025

Khởi động toàn bộ dịch vụ
docker-compose up -d --build

Kiểm tra trạng thái
docker-compose ps

Xem nhật ký hệ thống
docker-compose logs -f

Các điểm truy cập:

- Giao diện người dùng: <http://localhost:8000>
- Backend: <http://localhost:8082>
- Kiểm tra sức khỏe hệ thống: <http://localhost:8082/actuator/health>

8.1.4. Triển khai thực tế (Production)

Hướng dẫn triển khai lên VPS:

bash
Sao chépChỉnh sửa
Trên máy chủ VPS
sudo apt update
sudo apt install docker.io docker-compose git

```
# Tải mã nguồn và triển khai
git clone https://github.com/diengbtvu/se-2025.git
cd se-2025

# Cấu hình môi trường production
cp .env.example .env
# Chỉnh sửa file .env với giá trị thực tế

# Triển khai
docker-compose -f docker-compose.prod.yml up -d --build
Biến môi trường cần thiết:
env
Sao chépChỉnh sửa
# Cơ sở dữ liệu
DB_HOST=your_db_host
DB_USERNAME=your_db_user
DB_PASSWORD=your_db_password

# JWT
JWT_SECRET=your_jwt_secret_key

# API Google
GOOGLE_DRIVE_CLIENT_ID=your_client_id
GOOGLE_DRIVE_CLIENT_SECRET=your_client_secret
GEMINI_API_KEY=your_gemini_api_key
```

8.1.5. Hướng dẫn khắc phục sự cố

Một số lỗi phổ biến:

Xung đột cổng:

```
bash
Sao chépChỉnh sửa
# Kiểm tra cổng đang sử dụng
netstat -tlnp | grep :8080
```

```
# Dừng tiến trình nếu cần
sudo kill -9 PID
```

Lỗi kết nối cơ sở dữ liệu:

```
bash
Sao chépChỉnh sửa
# Kiểm tra kết nối MySQL
mysql -h localhost -u beelife_user -p beelifeventures
```

```
# Kiểm tra dịch vụ MySQL
sudo systemctl status mysql
```

Lỗi bộ nhớ:

bash

Sao chépChỉnh sửa

Kiểm tra dung lượng bộ nhớ

free -h

Tăng giới hạn bộ nhớ Docker

docker-compose up -d --memory=2g

Lỗi build ứng dụng:

bash

Sao chépChỉnh sửa

Dọn cache Maven

mvn clean

Xóa cache npm

npm cache clean --force

Xóa thư mục node_modules và cài lại

rm -rf node_modules && npm install

8.2. GitHub Repository & Liên kết bản demo

8.2.1. Kho lưu trữ GitHub (GitHub Repository)

Kho chính (Main Repository):

- URL: <https://github.com/diengbtvu/se-2025>
- Nhánh chính: main (sẵn sàng để triển khai)
- Giấy phép: Giấy phép MIT (MIT License)

Cấu trúc Repository (Repository Structure):

```
se-2025/
├── .github/workflows/    # CI/CD configurations
├── be/                  # Backend Spring Boot application
├── fe/                  # Frontend Next.js application
├── docs/                # Project documentation
├── scripts/             # Deployment scripts
├── docker-compose.yml    # Docker orchestration
└── README.md            # Project overview
```

Các nhánh chính (Key Branches):

- main: Mã nguồn sẵn sàng cho môi trường production
- develop: Nhánh tích hợp phát triển
- feature/*: Các nhánh phát triển tính năng riêng biệt

8.2.2. Liên kết demo trực tiếp (Live Demo Links)

Ứng dụng bản chính thức (Production Application):

- Giao diện người dùng (Frontend Demo): <https://beelife.zettix.net>
- Tài liệu API (API Documentation): <https://api.zettix.net/swagger-ui/index.html>
- Bảng điều khiển quản trị (Admin Dashboard): <https://beelife.zettix.net/admin>
(đã cung cấp tài khoản demo)

Thông tin đăng nhập demo (Demo Credentials):

Người dùng thông thường (Regular User):

- Tên đăng nhập: demouser
- Mật khẩu: demo123

Quản trị viên (Admin User):

- Tên đăng nhập: admin
- Mật khẩu: admin123

Các tính năng bản demo hỗ trợ (Demo Features Available):

- Duyệt danh mục sản phẩm với hình ảnh 3D
- Thêm sản phẩm vào giỏ và thực hiện thanh toán
- Theo dõi và quản lý đơn hàng
- Bảng điều khiển quản trị với số liệu phân tích
- Tương tác với chatbot AI
- Kiểm thử thiết kế responsive trên nhiều thiết bị

8.2.3. Liên kết tài liệu (Documentation Links)

Tài liệu kỹ thuật (Technical Documentation):

- Tham chiếu API: <https://api.zettix.net/swagger-ui/index.html>
- Storybook Frontend: (nếu có triển khai)
- Sơ đồ kiến trúc: Có sẵn trong thư mục /docs

Tài nguyên phát triển (Development Resources):

- Hướng dẫn cài đặt: README.md trong repository
- Hướng dẫn đóng góp: CONTRIBUTING.md
- Quy tắc ứng xử: CODE_OF_CONDUCT.md

8.2.4. Giám sát & Phân tích (Monitoring & Analytics)

Giám sát ứng dụng (Application Monitoring):

- **Kiểm tra trạng thái:** Tự động giám sát trạng thái ứng dụng
- **Chỉ số hiệu năng:** Thời gian phản hồi và tỷ lệ lỗi
- **Phân tích sử dụng:** Hành vi người dùng và mức độ sử dụng tính năng

Trạng thái CI/CD (CI/CD Status):

- **Trạng thái build:** *(hiển thị nếu có)*
- **Mức độ bao phủ mã nguồn:** Duy trì trên 80%
- **Kiểm tra bảo mật:** Tự động quét lỗ hổng bảo mật

8.2.5. Hỗ trợ & Liên hệ (Support & Contact)

Nhóm phát triển (Development Team):

- **Trưởng nhóm:** Trần Minh Điền – diengbtvu@gmail.com
- **Lập trình Backend:** Nguyễn Văn Hoàng – hoangvantvu@gmail.com
- **Lập trình Frontend:** Nguyễn Lê Duy – duynlfe@gmail.com

Kênh liên lạc dự án (Project Communication):

- **Vấn đề (Issues):** Sử dụng GitHub Issues để báo lỗi và yêu cầu tính năng
- **Thảo luận (Discussions):** Sử dụng GitHub Discussions cho các câu hỏi kỹ thuật
- **Tài liệu (Documentation):** Sử dụng Wiki pages cho hướng dẫn chi tiết

TÀI LIỆU THAM KHẢO

- [1] R. S. Pressman và B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., New York, NY, USA: McGraw-Hill Education, 2020.
- [2] I. Sommerville, *Software Engineering*, 10th ed., Boston, MA, USA: Pearson, 2016.
- [3] C. Walls, *Spring in Action*, 6th ed., Shelter Island, NY, USA: Manning Publications, 2022.
- [4] H. V. Thuận, *Giáo trình Kỹ nghệ phần mềm (Software Engineering)*, Hà Nội, Việt Nam: NXB Đại học Kinh tế Quốc dân, 2021.